

Relaxed Vector Commitment for Shorter Signatures

Seongkwang Kim¹, Byeonghak Lee¹, and Mincheol Son^{2*}

¹ Samsung SDS, Seoul, Korea
sk39.kim@samsung.com, byghak.lee@samsung.com

² KAIST, Daejeon, Korea
encrypted.def@kaist.ac.kr

Abstract. MPC-in-the-Head (MPCitH) has recently gained traction as a foundation for post-quantum signature schemes, offering robust security without trapdoors. Despite its strong security profile, MPCitH-based schemes suffer from high computational overhead and large signature sizes, limiting their practical application.

This work addresses these inefficiencies by relaxing vector commitments within MPCitH-based schemes. We introduce the concept of *vector semi-commitment*, which relaxes the binding property of traditional vector commitment. Vector semi-commitment schemes may allow an adversary to find more than one preimage of a commitment. We instantiate vector semi-commitment schemes in both the random oracle model and the ideal cipher model, leveraging recent optimizations on GGM tree such as correlated GGM tree.

We apply the ideal-cipher-based vector semi-commitment scheme to the BN++ signature scheme and prove it almost fully secure in the ideal cipher model. Implementing these improvements in the AImEr v2.0 signature scheme, we achieve up to 8.7% shorter signatures and up to 112% faster signing and verification speeds, setting new benchmarks for MPCitH-based schemes.

Keywords: MPC-in-the-Head, vector commitment, GGM tree, zero-knowledge proof, digital signature

Publication History. This paper is a revised version of Eurocrypt 2025 paper having the same name. In the proceeding version, the proposed signature scheme includes correlated GGM trees with secret key root which reduces $\tau\lambda$ bits. Kose and Xagawa pointed out that G_6 (reduction to the MIH game) in EUF-CMA security proof is wrong in private communication [26], and published a related paper [25]. We admitted that our security proof is wrong, and removed “the secret key root” from the signature scheme while maintaining cGGM itself.

* Mincheol Son was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No.RS-2024-00399491, Development of Privacy-Preserving Multiparty Computation Techniques for Secure Multiparty Data Integration)

So, in this version, we removed the cGGM-related technique which uses sk as a fixed root of GGM tree, and revised the corresponding contents. Since this technique reduces the signature size by $\tau\lambda$ bits, the signature scheme in this version has $\tau\lambda$ bits larger than the previous version. The revised contents include: definition of MIH, security proofs, signature sizes, added Δsk_k in algorithms, and some notations. For the performance figures of **rAIMer**, we did not update the previous figures for the following reason; as the updated **rAIMer** requires one more AES evaluation per view, the effect on the performance would not be significant because of optimization techniques such as pipelined evaluation of AES.

1 Introduction

The MPC-in-the-Head (MPCitH) paradigm [20] has recently emerged as a promising approach for designing post-quantum signature schemes. This paradigm leverages the concept of multi-party computation (MPC) to perform computations within a single entity’s “head”, and has been applied to zero-knowledge proofs and signature schemes. MPCitH-based signature schemes enable a signer to generate a signature without relying on a trapdoor, making their security depends solely on the one-way function used in key generation. This advantage allows primitives without trapdoors, such as block ciphers [31,11,24], unstructured multivariate quadratic (MQ) problem [13], or unstructured syndrome decoding problem [14] to base the security of signature schemes. This makes them more reliable compared to schemes whose security is based on artificially constructed hardness assumptions with potential gaps in the security reduction.

Despite their promising security feature, MPCitH-based signature schemes have been hindered by relatively high computational overhead and large signature sizes, making them less efficient compared to their lattice-based counterparts. The inherent complexity of simulating multi-party computations result in quadratic time and signature size with respect to the security parameter, which can be detrimental to practical adoption. In response, many studies have focused on optimizing the efficiency of MPCitH-based signature schemes through protocol optimization [23,6,30,1] and improved cryptographic primitives [11,24].

One notable line of work for improving the efficiency of MPCitH-based signature schemes is to optimize the GGM (Goldreich-Goldwasser-Micali) tree or vector commitment in the context of VOLE-in-the-Head (VOLEitH) [4] which includes the GGM tree and subsequent commitments afterward, a component used to generate shares of the virtual parties. The GGM tree enables the secure distribution of shares among the virtual parties. However, traditional GGM tree construction can be computationally expensive, contributing significantly to the overall inefficiency of MPCitH-based schemes. To address this, researchers developed more efficient GGM tree constructions, such as double-length PRG instantiated by fixed-key block cipher [9,8] and the application of correlated GGM technique [10,9,19].

1.1 Our Contribution

In this work, we focus on improving vector commitments used in MPCitH-based signature schemes. Our primary enhancement is to relax the vector commitment requirements. A vector commitment scheme, a crucial component of our MPCitH-based signature scheme, must satisfy two key properties: hiding and (extractable) binding. The hiding property ensures that the commitment conceals the committed values, safeguarding the data’s secrecy. The binding property ensures that once a commitment is made, it is computationally infeasible to alter the committed values without detection. Notably, a violation of the binding property does not directly result in a signature forgery.

We introduce a relaxed version of vector commitment, called *vector semi-commitment*, which ensures the hardness of finding a preimage and finding “many” collisions of a commitment. We call the latter characteristic *extractable semi-binding* – an extractor (only defined in the security proof) cannot find multi-collisions of a commitment more than a specific amount – and it is a key factor for the security proof.

We then instantiate vector semi-commitment schemes in both the random oracle model and the ideal cipher model. For the latter, we fully instantiate all the random primitives using ideal cipher calls except a constant number of hash function calls, incorporate recent optimizations of the GGM tree construction [9,8,19]. Previous works on optimizing the GGM tree can be divided into three contributions: the use of fixed-key block cipher, reducing primitive calls, and correlated GGM tree. While none of previous works include all the desirable characteristics, our vector semi-commitment scheme include all the above characteristics; ours calls $(N - 1)$ fixed-key block cipher for evaluating the GGM tree. By slightly modifying the Davies-Mayer construction provided in [8], our vector semi-commitment scheme enjoys better performance of both the fixed-key block cipher and the correlated GGM tree.

To see the practical implication of our vector semi-commitment, we apply all the improvements to BN++ [22], an MPCitH-based signature scheme. We prove its security up to $O(2^\lambda)$ queries to the ideal primitives in the ideal cipher model. In other words, we prove that replacing a vector commitment with a vector semi-commitment does not compromise the security of the resulted signature scheme. Performance-related parameters, such as the number of repetitions, remain unchanged, thereby reducing the signature size while maintaining performance.

We also implement our improvements in AImMer v2.0 [27], achieving up to 8.7% shorter signatures and up to 112% faster signing and verification speeds compared to AImMer v2.0. Compared to other MPCitH-based signature schemes such as SDitH [29] and FAEST [3], it also offers the fastest performance and the shortest signature size. Detailed performance figures are summarized in Table 2.

1.2 Related Work

In MPCitH-based signature schemes, a prover emulates an MPC protocol among N parties “in her head” and then opens the views of $(N - 1)$ parties except one.

The verifier accepts if all the views are consistent with an honest execution of the MPC protocol. Katz et al. employed the GGM tree to reduce the number of opened random seeds from $N - 1$ to $\log N$, subsequently applying it to the MPCitH-based signature scheme Picnic [23]. Since then, the GGM tree has been used as a core technique to reduce the signature size in the MPCitH-based signature schemes [11, 24, 14, 29, 7].

There have been efforts to improve GGM trees. Guo et al. proposed a correlated GGM tree [17] in the context of correlated oblivious transfer and distributed point function. In a correlated GGM tree, the sum of all nodes at the same level is fixed. This technique reduces the number of random permutation calls by half. The correlated GGM tree was recently applied VOLE-in-the-Head-based signature schemes. Cui et al. reduces the number of random permutation calls for generating seeds is halved [10], and Huth and Joux reduces the signature size by deleting the correction to the secret key share. Concurrently, Bui and Cong proposed applying the correlated GGM tree to MPC-in-the-Head and VOLE-in-the-Head [9]. They also replaced the random oracle calls used for commitment with random permutation calls, which can be implemented using efficient primitives such as fixed-key AES. However, the provable security of their proposal did not exceed the birthday bound. Independent with the correlated GGM tree, Bui et al. proposed a fast salted GGM tree which is fully secure in the ideal cipher model [8]. Using this salted tree, tree evaluation is as fast as the best unsalted version while preventing multi-target attacks. Compared to these works, our vector semi-commitment enjoys all the good characteristics: fast, salted, half number of calls, reduced signature size, and full security with respect to the number of ideal primitive queries.

2 Preliminaries

2.1 Notations

For two vectors or strings a and b , their concatenation is denoted by $a||b$. For a vector or string a and non-negative integer m , we denote first m element of a by $a[:m]$. For integers a and b , we denote the bitwise XOR of a and b by $a \oplus b$. Bitwise right shift by i of a is denoted by $a \gg i$.

We denote $[n] = \{1, \dots, n\}$. For a vector $\mathbf{x}$, $\mathbf{x}[a:b]$ denotes a slice of the vector $\mathbf{x}$ from index a (inclusive) to b (inclusive). $\mathbf{x}[:b]$ and $\mathbf{x}[a:]$ denotes the same as $\mathbf{x}[1:b]$ and $\mathbf{x}[a:\ell]$ respectively, where ℓ refers to the length of the vector $\mathbf{x}$. Unless stated otherwise, all logarithms are to the base 2. For an integer $a \in \{0, 1, \dots, 255\}$, $\langle a \rangle_B$ is the canonical binary representation of a , which is an 8-bit string. For a positive integer n and $k < n$, the falling factorial is denoted by $(n)_k = n \cdot (n-1) \cdot \dots \cdot (n-k+1)$.

For a set S , we write $a \leftarrow_{\$} S$ to denote that a is chosen uniformly at random from S . For a probability distribution $\mathcal{D}$, $a \leftarrow \mathcal{D}$ denotes that a is sampled according to the distribution $\mathcal{D}$. We denote the binomial distribution with n trials and probability p by $B(n, p)$.

Throughout this paper, the security parameter is denoted by λ and assume $\lambda \geq 64$. In the multiparty computation setting, $x^{(i)}$ denotes the i -th party's additive share of x , implying that $\sum_i x^{(i)} = x$.

2.2 Chernoff Bound

Chernoff bound is a well-known upper bound on the tail of a random variable. As our proof relies on this upper bound, we briefly introduce the Chernoff bound in its multiplicative form.

Let X be a random variable following the binomial distribution $B(Q, p)$. Then, the probability of the tail of X is upper bounded by

$$\Pr[X \geq (1 + \delta)\mu] \leq \left(\frac{e^\delta}{(1 + \delta)^{1+\delta}} \right)^\mu < \frac{1}{e^\mu} \cdot \left(\frac{e\mu}{(1 + \delta)\mu} \right)^{(1+\delta)\mu}$$

for any $\delta > 0$, where e is Euler's number and $\mu = Q \cdot p$ is the expectation of X .

Given a multiset $S = \{x_1, x_2, \dots, x_Q\}$ with $x_i \leftarrow_{\$} \{0, 1\}^\lambda \setminus \mathcal{D}_i$ where $|\mathcal{D}_i| \leq 2^{\lambda-1}$, we can bound the probability that the maximum number of multi-collision mcoll in S is greater than $2\lambda/\log \lambda$ as follows. With $\lambda \geq 64$ and $Q \leq 2^{\lambda-1}$, the probability is bounded by

$$\begin{aligned} \Pr[\text{mcoll} \geq 2\lambda/\log \lambda] &< \left(\frac{e \log \lambda}{2\lambda} \cdot \frac{2Q}{2^\lambda} \right)^{2\lambda/\log \lambda} \cdot 2^\lambda \leq \left(\frac{e \log \lambda}{\lambda^{1/2}} \cdot \frac{Q}{2^\lambda} \right)^{2\lambda/\log \lambda} \\ &\leq \left(\frac{2e}{3} \cdot \frac{Q}{2^\lambda} \right)^{2\lambda/\log \lambda} \leq \frac{2Q}{2^\lambda}. \end{aligned} \quad (1)$$

Similarly, given a multiset $S' = \{y_1, \dots, y_Q\}$ with $y_i \leftarrow_{\$} \{0, 1\}^c \setminus \mathcal{D}'_i$ where $|\mathcal{D}'_i| \leq 2^{c-1}$, we can bound the maximum multi-collision mcoll' in S' is greater than 13 as follows. With $Q^6 \leq 2^{5c}$,

$$\Pr[\text{mcoll}' \geq 13] < \left(\frac{2eQ}{13 \cdot 2^c} \right)^{13} \cdot 2^c \leq \frac{1}{2^{16}} \cdot \frac{Q}{2^{2c}}. \quad (2)$$

2.3 GGM Tree

GGM tree is a binary tree proposed by Goldreich, Goldwasser, and Micali [16]. For a power-of-two integer N , one can send $N - 1$ out of N random strings with $\log N$ communication by using a GGM tree. For a pseudorandom generator $G : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ and a root node_0 , the nodes in a GGM tree are defined recursively as follows.

$$\begin{aligned} \text{node}_{1,1} \parallel \text{node}_{1,2} &= G(\text{node}_0) \\ \text{node}_{i,2j-1} \parallel \text{node}_{i,2j} &= G(\text{node}_{i-1,j}) \quad \text{for } i \geq 2 \text{ and } 0 < j \leq 2^{i-1} \end{aligned}$$

Let a GGM tree $\mathcal{T}$ has 2^d leaf nodes. If one wants to send all leaf nodes except k -th leaf node (i.e., $\text{node}_{d,k}$), she can send a Merkle path

$$(\text{node}_{1,((k-1) \gg (d-1)) \oplus 1 + 1}, \text{node}_{2,(((k-1) \gg (d-2)) \oplus 1) + 1}, \dots, \text{node}_{d,((k-1) \oplus 1) + 1}).$$

We call this Merkle path an *associate path* of the unopened node $\text{node}_{d,k}$. Conversely, we will call the unopened node $\text{node}_{d,k}$ an *associate node* of the Merkle path described above. The depth of a node is defined by the length of the shortest upward path to the root (e.g., the depth of $\text{node}_{1,1}$ is 1), and the height of a node is defined by the length of the longest downward path to a leaf node.

2.4 BN++ Zero-knowledge Protocol

In this section, we briefly review the BN++ proof system [22], one of the state-of-the-art MPCitH-based zero-knowledge protocols. At a high level, BN++ is a variant of the BN protocol [5] with several optimization techniques applied to reduce the signature size.

PROTOCOL OVERVIEW. BN++ essentially simulates multiparty computation of *triple checking protocol*, which verifies that all the multiplication triples are honestly generated. To check C multiplication triples $(x_j, y_j, z_j = x_j \cdot y_j)_{j=1}^C$ over a finite field $\mathbb{F}$ in the multiparty computation setting with N parties, *helping values* $((a_j, b_j)_{j=1}^C, c)$ are required, where $a_j \leftarrow \mathbb{F}$, $b_j = y_j$, and $c = \sum_{j=1}^C a_j \cdot b_j$. Each party holds secret shares of the multiplication triples $(x_j, y_j, z_j)_{j=1}^C$ and helping values $((a_j, b_j)_{j=1}^C, c)$. Then, the protocol proceeds as follows.

- A prover is given random challenges $\epsilon_1, \dots, \epsilon_C \in \mathbb{F}$.
- For $i \in [N]$, the i -th party locally sets $\alpha_1^{(i)}, \dots, \alpha_C^{(i)}$ where $\alpha_j^{(i)} = \epsilon_j \cdot x_j^{(i)} + a_j^{(i)}$.
- The parties open $\alpha_1, \dots, \alpha_C$ by broadcasting their shares.
- For $i \in [N]$, the i -th party locally sets

$$v^{(i)} = \sum_{j=1}^C \epsilon_j \cdot z_j^{(i)} - \sum_{j=1}^C \alpha_j \cdot b_j^{(i)} + c^{(i)}.$$

- The parties open v by broadcasting their shares and output **Accept** if $v = 0$.

By Lemma 1, the probability that there exist incorrect triples and the parties output **Accept** in a single run of the above steps is upper bounded by $1/|\mathbb{F}|$.

Lemma 1 ([22]) *If the secret-shared input $(x_j, y_j, z_j)_{j \in [C]}$ contains an incorrect multiplication triple, or if the shares of $((a_j, b_j)_{j \in [C]}, c)$ form an incorrect dot product, then the parties output **Accept** in the sub-protocol with probability at most $1/|\mathbb{F}|$.*

SIGNATURE SIZE. By applying the Fiat-Shamir transform [12], one can obtain a signature scheme from the BN++ proof system. In this signature scheme, the signature size is given as

$$6\lambda + \tau \cdot (3\lambda + \lambda \cdot \lceil \log_2(N) \rceil + \mathcal{M}(C)),$$

where λ is the security parameter, C is the number of multiplication gates in the underlying symmetric primitive, and $\mathcal{M}(C) = (2C + 1) \cdot \log_2(|\mathbb{F}|)$. In particular, $\mathcal{M}(C)$ is defined from the observation that sharing the secret share offsets for $(z_j)_{j=1}^C$ and c , and opening shares for $(\alpha_j)_{j=1}^C$ occurs for each repetition, using C , 1, and C elements of $\mathbb{F}$, respectively.

2.5 H-coefficient Technique

The H-coefficient technique is a powerful method used in the analysis of cryptographic algorithms, particularly in the context of provable security. Introduced by Patarin, this technique provides a systematic way to bound the distinguishing advantage of an adversary interacting with an idealized cryptographic system and a real implementation. The core idea of the H-coefficient technique is to partition the set of possible transcripts (i.e., sequences of queries and responses) into two subsets: “good” and “bad” transcripts. The probability of bad transcripts can be shown to be negligible, while the probability of distinguishing between the distributions of good transcripts can be tightly bounded.

To illustrate the H-coefficient technique, we present the following lemma, which is a fundamental component of the technique:

Lemma 2 (H-Coefficient Lemma) *Let $\mathcal{A}$ be an algorithm that interacts with either an ideal world or a real world and tries to distinguish two worlds. Let T_{id} and T_{re} be the distribution of transcript in the ideal world and the real world, respectively, and $\mathcal{T}$ denotes the set of all attainable transcripts in the ideal world. Suppose there exist partition $\mathcal{T}_{\text{Good}}$ (good transcripts) and $\mathcal{T}_{\text{Bad}}$ (bad transcripts) of $\mathcal{T}$, and constants ϵ and δ such that for any $\gamma \in \mathcal{T}_{\text{Good}}$,*

$$\Pr[T_{\text{id}} \in \mathcal{T}_{\text{Bad}}] \leq \epsilon, \quad \frac{\Pr[\gamma = T_{\text{re}}]}{\Pr[\gamma = T_{\text{id}}]} \geq 1 - \delta.$$

Then, the distinguishing advantage of $\mathcal{A}$ in distinguishing $\mathcal{I}$ from $\mathcal{R}$ is bounded by:

$$\text{Adv}_{\mathcal{I}, \mathcal{R}}^{\text{dist}}(\mathcal{A}) \leq \epsilon + \delta.$$

This lemma provides a clear framework for analyzing the security of cryptographic protocols. By carefully defining the sets of good and bad transcripts and bounding their probabilities, one can apply the H-coefficient technique to obtain rigorous security proofs.

3 Vector Semi-commitment

In the context of MPCitH-based signature schemes, vector commitment (VC) abstracts the process of generating views and corresponding commitments. A VC scheme typically consists of four algorithms: **Commit**, **Open**, **Recon**, and **Verify**. The **Commit** algorithm generates the views of virtual parties and commits to these views. When a verifier challenges which views to reveal, the prover uses **Open** to disclose a subset of the views. **Open** produces partial decommitment information. Using this partial decommitment information, the verifier runs **Recon** to reconstruct the revealed messages and the corresponding commitments. Finally, the verifier uses **Verify** to check the validity of the partial decommitment information.

A vector semi-commitment scheme shares the same interface as vector commitments, with one of its properties relaxed compared to traditional vector commitments. We introduce the interface of vector semi-commitment in the random oracle model. Although the following is described in the context of the random oracle model, it can be easily adapted to the ideal cipher model.

Definition 1 (Vector Semi-commitment). *Let H be a random oracle. An IV-based vector commitment scheme VSC with message space $\mathcal{M}$ in a random oracle model is defined by the following PPT algorithms.*

- $\text{Commit}^H(\text{salt}, \text{root}) \rightarrow (\text{com}, \text{decom}, (m_1, \dots, m_N))$: given an IV salt and a λ -bit string root, output a commitment com with opening information decom for messages $m = (m_1, \dots, m_N) \in \mathcal{M}^N$.
- $\text{Open}^H(\text{salt}, \text{decom}, I) \rightarrow \text{pdecom}$: given an IV salt, opening information decom and a subset $I \subseteq [N]$ of indices, output a partial opening information pdecom for I .
- $\text{Recon}^H(\text{salt}, \text{pdecom}, I) \rightarrow ((m_i)_{i \in I}, \text{com})$: given an IV salt, a partial opening information pdecom for a subset I , output partially reconstructed messages and the full commitment com.
- $\text{Verify}^H(\text{salt}, \text{com}, \text{pdecom}, I) \rightarrow \{(m_i)_{i \in I}\} \cup \{\perp\}$: given an IV salt, a commitment com, a partial opening information pdecom, and a subset I , either output the messages $(m_i)_{i \in I}$ (accept) or $\perp$ (reject).

Although the interface is written for a general subset I , we will primarily consider I as a subset missing a single element unless otherwise specified. This opening is referred to as an *all-but-one*, with I being the all-but-one subset. If the context is clear, we will omit the random oracle indication (H).

The major difference between vector semi-commitment and vector commitment is the binding property. Informally, the binding property of a commitment implies that it is hard to find multiple messages corresponding to the same commitment. A vector semi-commitment has a relaxed version of this property: the extractable semi-binding property. This property implies that while an adversary may find a small number of messages corresponding to the same commitment, it is hard to find a large number of such messages.

Definition 2 (Extractable Semi-binding). *Let VSC be an IV-based vector semi-commitment scheme in the random oracle model with random oracle H . Let $\text{Ext}(\text{salt}, \mathcal{Q}, \text{com}) \rightarrow \{(m_i^{(j)})_{i \in [N]}\}_{j \in J}$ be a PPT algorithm that, given a set of query-response pairs of random oracle queries $\mathcal{Q}$ and a commitment com, outputs the committed messages $(m_i)_{i \in [N]}$. The u -extractable semi-binding game for VSC, denoted as $u\text{-ESB}$, with $N = \text{poly}(\lambda)$ and Q queries to the random oracle and stateful $\mathcal{A}$, is defined as follows.*

1. $(\text{salt}, \text{com}, \text{pdecom}_I, (m_i)_{i \in I}, I) \leftarrow \mathcal{A}(1^\lambda, Q)$
2. $\{(m_i^{(j)})_{i \in [N]}\}_{j \in J} \leftarrow \text{Ext}(\text{salt}, \mathcal{Q}, \text{com})$, where $\mathcal{Q}$ is the set $\{(x_i, H(x_i))\}$ of query-response pairs of queries $\mathcal{A}$ made to H and $|J| \leq u$.

3. Output 1 if $\text{Verify}(\text{iv}, \text{com}, \text{pdecom}_I, I) \rightarrow (m_i^*)_{i \in I}$ but there no $j \in J$ such that $m_i^* = m_i^{(j)}$ for all $i \in I$; otherwise, output 0.

We define $\mathcal{A}$'s u -extractable semi-binding advantage by

$$\text{Adv}_{\text{VSC}}^{u\text{-esb}}(\mathcal{A}) = \Pr[\mathcal{A} \text{ wins } u\text{-ESB}].$$

Although the extractor Ext extracts the set of messages $(m_i^{(j)})_{i,j}$, the purpose of the extractable semi-binding game is to bound the number of valid pdecom for the same com . Therefore, the extractor should be programmed to extract messages only from valid pdecom 's. We note that Ext may output $m_i = \perp$ if the committed value at index i is invalid.

The next property of vector semi-commitment is hiding, but the following definition is in the multi-instance form, and take into account the correlated property. The intuition behind the multi-instance hiding is that given multiple puncturable PRF instances and their commitments, the punctured messages are indistinguishable from random.

Definition 3 (Multi-Instance Hiding). Let VSC be an IV -based vector semi-commitment scheme in the random oracle model with random oracle H . The multi-instance hiding game for vector semi-commitments, $Q_I\text{-MIH}$, with $N = \text{poly}(\lambda)$ and Q queries to the random oracle and stateful $\mathcal{A}$ is defined as follows.

1. $b^* \leftarrow_{\$} \{0, 1\}$
2. For $j \in [Q_I]$, do the following:
 - (a) $\text{root}_j \leftarrow_{\$} \{0, 1\}^\lambda$
 - (b) $\text{salt}_j \leftarrow_{\$} \{0, 1\}^{2\lambda+8}$
 - (c) $(\text{com}_j, \text{decom}_j, (m_{j,1}^*, \dots, m_{j,N}^*)) \leftarrow \text{Commit}(\text{salt}_j, \text{root}_j)$
 - (d) $\bar{i}_j \leftarrow_{\$} [N]$, $I_j = [N] \setminus \{\bar{i}_j\}$
 - (e) $\text{pdecom}_j \leftarrow \text{Open}(\text{salt}_j, \text{decom}_j, I_j)$
 - (f) $m_{j,i} \leftarrow m_{j,i}^*$ for $i \in I_j$.
 - (g) Set $m_{j,\bar{i}_j} \leftarrow \begin{cases} m_{j,\bar{i}_j}^* & \text{if } b^* = 0, \\ \leftarrow_{\$} \mathcal{M} & \text{otherwise.} \end{cases}$
3. $b \leftarrow \mathcal{A}^H(\text{salt}_j, (m_{j,i})_{i \in [N]}, \text{pdecom}_j, I_j)_{j \in [Q_I]}$.
4. Output 1 (win) if $b = b^*$, else 0.

We define $\mathcal{A}$'s multi-instance hiding advantage by

$$\text{Adv}_{\text{VSC}}^{Q_I\text{-mih}}(\mathcal{A}) = \left| \Pr[\mathcal{A} \text{ wins } Q_I\text{-MIH}] - \frac{1}{2} \right|.$$

We call VSC is multi-instance hiding if advantage of any $Q_I\text{-MIH}$ adversary for polynomially bounded Q_I is negligible.

3.1 Instantiation from Random Oracle

In this section, we instantiate VSC from a random oracle, dubbed RO-VSC, and prove the extractable semi-binding property and multi-instance hiding property. Let $H_{\text{com}} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, $H_{\text{tree}} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, and $H_{\text{exp}} : \{0, 1\}^* \rightarrow \mathbb{F}^{2C+2}$ be random oracles. Then, RO-VSC is constructed as Figure 1.

The commitment process **Commit** involves an evaluation of the correlated GGM tree to generate seeds and tapes, culminating in the output of a commitment **com** and opening information **decom**. Compared to the traditional vector commitment based on the GGM tree, our RO-VSC employs the correlated GGM tree, and the commitment size is reduced from 2λ to λ . The double-length PRG part (Step 2 in **Commit**^H) of RO-VSC is depicted in fig. 2a. Other algorithms are similar to vector commitment schemes. The opening algorithm **Open** extracts path information for a given index set, while the reconstruction algorithm **Recon** rebuilds messages and commitments from path data and verifies the integrity of the commitment. Finally, the verification algorithm **Verify** ensures that the reconstructed messages match the original commitment. The integer C corresponds to the number of multiplication gates, and the integer N corresponds to the number of parties in the signature scheme.

As this work focuses on security proof in the ideal cipher model, we only state Lemma 3 for extractable semi-binding property and Lemma 4 for multi-instance hiding property of RO-VSC. The proofs of the lemmas are provided in Supplementary Material B.1.

Lemma 3 *Let $H_{\text{com}} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ and $H_{\text{tree}} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ be random oracles. Let $\mathcal{A}$ be an arbitrary adversary that makes Q queries to the random oracles. Then $\mathcal{A}$'s u -extractable semi-binding advantage $\text{Adv}_{\text{RO-VSC}}^{u\text{-esb}}(\mathcal{A})$ against RO-VSC is bounded by*

$$\text{Adv}_{\text{RO-VSC}}^{u\text{-esb}}(\mathcal{A}) \leq \frac{10Q}{2^\lambda},$$

$$\text{for } u = 2N \left(\frac{\lambda}{\log \lambda} \right)^2.$$

Lemma 4 *Let $H_{\text{com}} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, $H_{\text{tree}} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, and $H_{\text{exp}} : \{0, 1\}^* \rightarrow \{0, 1\}^*$ be random oracles. Let $\mathcal{A}$ be an arbitrary adversary that makes Q queries to the random oracles. Then $\mathcal{A}$'s multi-instance hiding advantage $\text{Adv}_{\text{RO-VSC}}^{Q_I\text{-mih}}(\mathcal{A})$ against RO-VSC is bounded by*

$$\text{Adv}_{\text{RO-VSC}}^{Q_I\text{-mih}}(\mathcal{A}) \leq \frac{Q_I^2}{2^{2\lambda}} + \frac{Q}{2^\lambda}.$$

3.2 Instantiation from Ideal Cipher

Now we replace the random oracles with an ideal cipher E . We instantiate VSC from the ideal cipher, named IC-VSC, and also prove its properties. Let $E :$

- **Parameters:** a triple of random oracles $H = (H_{\text{com}}, H_{\text{tree}}, H_{\text{exp}})$, an integer C , a power-of-two integer $N = 2^d$.
- **Inputs:** $\text{salt} \in \{0, 1\}^{2\lambda+8}$, and $\text{root} \in \{0, 1\}^\lambda$.
- $\text{Commit}^H(\text{salt}, \text{root})$:
 1. Set $\text{node}_{0,1} \leftarrow \text{root}$.
 2. For each level $e \in [d]$ and $i \in [2^e]$, set

$$\begin{aligned} \text{node}_{e,2i-1} &\leftarrow H_{\text{tree}}(\text{salt}, e, i, \text{node}_{e-1,i}) \\ \text{node}_{e,2i} &\leftarrow H_{\text{tree}}(\text{salt}, e, i, \text{node}_{e-1,i}) \oplus \text{node}_{e-1,i}. \end{aligned}$$
 3. For $i \in [N]$, set

$$\begin{aligned} \text{seed}_i &\leftarrow \text{node}_{d,i} \\ \text{com}_i &\leftarrow H_{\text{com}}(\text{salt}, i, \text{seed}_i) \\ \text{tape}_i &\leftarrow H_{\text{exp}}(\text{salt}, i, \text{seed}_i) \\ m_i &= \text{tape}_i \end{aligned}$$
 4. Output a commitment $\text{com} = (\text{com}_1, \dots, \text{com}_N)$ with opening information $\text{decom} = ((\text{node}_{e,i})_{e \in [d-1], i \in [2^e]}, \text{com})$, and messages $(m_1, \dots, m_N)$.
- $\text{Open}^H(\text{salt}, \text{decom}, I = [N] \setminus \{\bar{i}\})$:
 1. Set $\text{path}_I \leftarrow (\text{node}_{d-e+1, i_e})_{e \in [d]}$ where $i_e = (\lfloor (\bar{i} - 1)/2^{e-1} \rfloor \oplus 1) + 1$ for $e \in [d]$.
 2. Output $\text{pdecom} = (\text{path}_I, \text{com}_{\bar{i}})$.
- $\text{Recon}^H(\text{salt}, \text{pdecom}, I = [N] \setminus \{\bar{i}\})$:
 1. Similarly as Step 2 in Commit, expand each node in path_I and get $(\text{node}_{d,i})_{i \in I}$.
 2. For $i \in I$, do Step 3 in Commit.
 3. Output $((m_i)_{i \in I}, \text{com})$.
- $\text{Verify}^H(\text{salt}, \text{com} = (\text{com}_i^*)_{i \in [N]}, \text{pdecom}, I = [N] \setminus \{\bar{i}\})$:
 1. Similarly as Step 2 in Commit, expand each node in path_I and get $(\text{node}_{d,i})_{i \in I}$.
 2. For $i \in I$, do Step 3 in Commit.
 3. Output $(m_i)_{i \in I}$ if $\text{com} = \text{com}_i^*$ for all $i \in I$, or output $\perp$ otherwise.

Fig. 1: RO-VSC

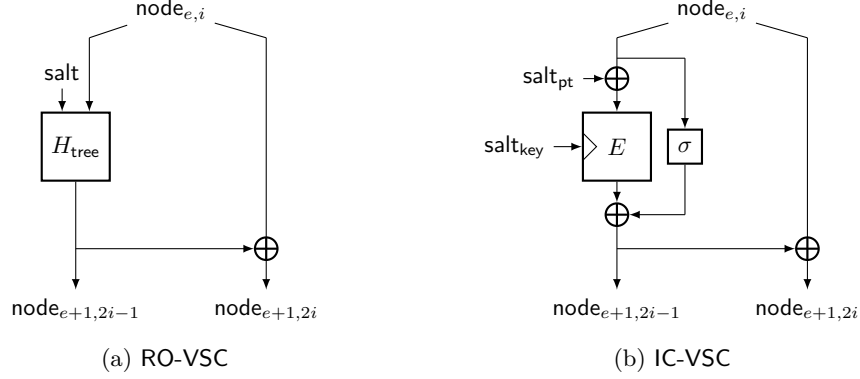


Fig. 2: Double-length PRG of RO-VSC and IC-VSC.

$\{0,1\}^\lambda \times \{0,1\}^\lambda \rightarrow \{0,1\}^\lambda$ be a ideal cipher, and $\sigma : \{0,1\}^\lambda \rightarrow \{0,1\}^\lambda$ be an linear orthomorphism. Then IC-VSC is constructed as Figure 3.

The main difference between IC-VSC and RO-VSC is that all the random oracles are replaced with the ideal cipher, which requires sophisticated domain separation. The GGM tree evaluation (Step 2 in Commit^E), the share generation (Step 3 in Commit^E) are realized using only ideal cipher calls. We note that the whole τ GGM trees in a signature can be evaluated by a fixed-key block cipher. The double-length PRG part of the GGM tree evaluation is depicted in Figure 2b. One noteworthy point is that the double-length PRG of IC-VSC is inspired by Davies-Meyer-based GGM tree proposed by Bui et al. [8], and it is modified to an orthomorphism-applied version to work properly in correlated GGM tree. The new variable $\mathbf{b}$ represents the current repetition in the signature scheme.

In the following, we prove the extractable semi-binding property and the multi-instance hiding property of IC-VSC with a supporting lemma. Although the proofs are straightforward, beware that the definition of Ext in Lemma 6 will be used in the EUF-KO security proof. Lemma 5 is a supporting lemma which limits the probability of commitment collision from distinct height-1 nodes (height-0 nodes mean leaf nodes, not commitments), so that there is only one height- ≥ 1 node for each position for a fixed commitment.

Lemma 5 *Let $E : \{0,1\}^\lambda \times \{0,1\}^\lambda \rightarrow \{0,1\}^\lambda$ be an ideal cipher and $\sigma : \{0,1\}^\lambda \rightarrow \{0,1\}^\lambda$ be an orthomorphism. Let $\mathcal{A}$ be an arbitrary adversary that makes Q queries to E . Then, the probability that $\mathcal{A}$ finds $\text{salt} = (\text{salt}_{\text{pt}}, \text{salt}_{\text{key}}, \mathbf{b}) \in \{0,1\}^{\lambda+\lambda+8}$, $i \in \{0, 2, \dots, 254\}$, and distinct $n, n' \in \{0,1\}^\lambda$ such that*

$$\begin{cases}
 E_{\sigma(n) \oplus E_{\text{salt}_{\text{key}}}(n \oplus \text{salt}_{\text{pt}})}(\text{salt}_{\text{pt}}[\lambda - 24] \parallel \mathbf{b} \parallel \langle i \rangle_B \parallel \langle 0 \rangle_B) \\
 = E_{\sigma(n') \oplus E_{\text{salt}_{\text{key}}}(n' \oplus \text{salt}_{\text{pt}})}(\text{salt}_{\text{pt}}[\lambda - 24] \parallel \mathbf{b} \parallel \langle i \rangle_B \parallel \langle 0 \rangle_B), \\
 E_{\sigma(n) \oplus E_{\text{salt}_{\text{key}}}(n \oplus \text{salt}_{\text{pt}}) \oplus n}(\text{salt}_{\text{pt}}[\lambda - 24] \parallel \mathbf{b} \parallel \langle i + 1 \rangle_B \parallel \langle 0 \rangle_B) \\
 = E_{\sigma(n') \oplus E_{\text{salt}_{\text{key}}}(n' \oplus \text{salt}_{\text{pt}}) \oplus n'}(\text{salt}_{\text{pt}}[\lambda - 24] \parallel \mathbf{b} \parallel \langle i + 1 \rangle_B \parallel \langle 0 \rangle_B),
 \end{cases} \quad (3)$$

- **Parameters:** an ideal cipher E , an orthomorphism σ , integer C , a power-of-two integer N , and $N = 2^d$.
- **Inputs:** $\text{salt} = (\text{salt}_{\text{pt}}, \text{salt}_{\text{key}}, \mathbf{b}) \in \{0, 1\}^{\lambda+\lambda+8}$, and $\text{root} \in \{0, 1\}^\lambda$.
- $\text{Commit}^E(\text{salt}, \text{root})$:
 1. Set $\text{node}_{0,1} \leftarrow \text{root}$.
 2. For each level $e \in [d]$ and $i \in [2^e]$, set

$$\begin{aligned} \text{node}_{e,2i-1} &\leftarrow \sigma(\text{node}_{e-1,i}) \oplus E_{\text{salt}_{\text{key}}}(\text{node}_{e-1,i} \oplus \text{salt}_{\text{pt}}) \\ \text{node}_{e,2i} &\leftarrow \text{node}_{e,2i-1} \oplus \text{node}_{e-1,i}. \end{aligned}$$
 3. For $i \in [N]$, set

$$\begin{aligned} \text{seed}_i &\leftarrow \text{node}_{d,i} \\ \text{ctr}[\mathbf{b}, i, 0] &\leftarrow \text{salt}_{\text{pt}}[: \lambda - 24] \parallel \mathbf{b} \parallel \langle i \rangle_B \parallel \langle 0 \rangle_B \\ \text{com}_i &\leftarrow E_{\text{seed}_i}(\text{ctr}[\mathbf{b}, i, 0]) \end{aligned}$$
 - (a) For $j \in [2C + 2]$,

$$\begin{aligned} \text{ctr}[\mathbf{b}, i, j] &\leftarrow \text{salt}_{\text{pt}}[: \lambda - 24] \parallel \mathbf{b} \parallel \langle i \rangle_B \parallel \langle j \rangle_B \\ \text{tape}_{i,j} &\leftarrow E_{\text{seed}_i}(\text{ctr}[\mathbf{b}, i, j]) \end{aligned}$$
 - (b) Set $\text{tape}_i \leftarrow \text{tape}_{i,1} \parallel \dots \parallel \text{tape}_{i,2C+2}$ and $m_i \leftarrow \text{tape}_i$.
 4. Output a commitment $\text{com} = (\text{com}_1, \dots, \text{com}_N)$ with opening information $\text{decom} = ((\text{node}_{e,i})_{e \in [d-1], i \in [N]}, \text{com})$, and messages $(m_1, \dots, m_N)$.
- $\text{Open}^E(\text{salt}, \text{decom}, I = [N] \setminus \{\bar{i}\})$:
 1. Set $\text{path}_I \leftarrow (\text{node}_{d-e+1, i_e})_{e \in [d]}$ where $i_e = (\lfloor (\bar{i} - 1)/2^{e-1} \rfloor \oplus 1) + 1$ for $e \in [d]$;
 2. Output $\text{pdecom} = (\text{path}_I, \text{com}_{\bar{i}})$
- $\text{Recon}^E(\text{salt}, \text{pdecom}, I = [N] \setminus \{\bar{i}\})$:
 1. Similarly as Step 2 in Commit, expand each node in path_I and get $(\text{node}_{d,i})_{i \in I}$.
 2. For $i \in I$, do Step 3 in Commit^E .
 3. Output $((m_i)_{i \in I}, \text{com})$.
- $\text{Verify}^E(\text{salt}, \text{com} = (\text{com}_i^*)_{i \in [N]}, \text{pdecom}, I = [N] \setminus \{\bar{i}\})$:
 1. Similarly as Step 2 in Commit, expand each node in path_I and get $(\text{node}_{d,i})_{i \in I}$.
 2. For $i \in I$, do Step 3 in Commit^E .
 3. Output $(m_i)_{i \in I}$ if $\text{com} = \text{com}_i^*$ for all $i \in I$, or output $\perp$ otherwise.

Fig. 3: IC-VSC

is at most $10Q/2^\lambda$.

Technically speaking, we limit the number of single commitment collision by Markov's inequality, and limit the probability of two adjacent commitments collisions. The intuition behind this so technical proof is that the total length of commitments assigned to a node with two leaves will be 2λ . Therefore, there will be no two distinct height-1 nodes with same commits. For the full proof, see Supplementary material B.2.

Lemma 6 *Let $E : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ be an ideal cipher and $\sigma : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ be an orthomorphism. Let $\mathcal{A}$ be an arbitrary adversary that makes Q queries to E . Then, $\mathcal{A}$'s u -extractable semi-binding advantage $\text{Adv}_{\text{IC-VSC}}^{u\text{-esb}}(\mathcal{A})$ against IC-VSC is bounded by*

$$\text{Adv}_{\text{IC-VSC}}^{u\text{-esb}}(\mathcal{A}) \leq \frac{14Q}{2^\lambda},$$

for $u = 2N \left(\frac{\lambda}{\log \lambda} \right)^2$.

Proof. Intuitively, according to Lemma 5, the probability of finding a collision in commitments derived by a non-leaf node is negligible. Furthermore, for each leaf node, the number of multi-collisions is bounded by the Chernoff bound. We now proceed to formally bound the adversary's advantage.

Let Q be the number of queries to E , and $\mathcal{Q}$ be the collection of all queries to E . Since the choice of salt and $\mathbf{b}$ does not affect extractable semi-binding, we can assume all salts and $\mathbf{b}$ are equaled to zero strings.

We first define the extractor $\text{Ext}(0^\lambda, \mathcal{Q}, \text{com} = (\text{com}_1, \dots, \text{com}_N))$ as follows.

1. For each $i \in [N]$, find leaf node sets $S_{d,i}$ such that

$$S_{d,i} = \{s : E_s(\text{ctr}[\langle 0 \rangle_B, i, 0]) = \text{com}_i\}$$

2. For each $e \in [d-1]$ and $i \in [2^e]$, find internal node sets $S_{e,i}$ such that

$$S_{e,i} = \{n : E_0(n) \oplus \sigma(n) \in S_{e+1,2i-1}, E_0(n) \oplus \sigma(n) \oplus n \in S_{e+1,2i}\}$$

3. For $i \in [N]$, let

$$A_i = \left\{ (p_1, \dots, p_d) : p_e \in S_{e,i_e} \text{ for } e \in [d] \right\}$$

where $i_e = (\lfloor (i-1)/2^{d-e} \rfloor \oplus 1) + 1$ for $e \in [d]$

4. Let S be the set of messages, where

$$S = \left\{ (s_1, \dots, s_N) : s_i = \perp \text{ if } S_{d,i} = \emptyset \text{ and } s_i \in S_{d,i} \text{ otherwise,} \right. \\ \left. \text{Recon}(p_1, \dots, p_d, \text{com}_i, I = [N] \setminus \{i\}) = (s_i)_{i \in I} \text{ for } (p_1, \dots, p_d) \in A_i \right\}$$

5. Finally, Ext outputs arbitrary u or less elements in S .

For graphical description, we make an example in Figure 4. For a fixed commitment com , $S_{e,i}$ is the set of i -th depth- e nodes whose corresponding leaf nodes are consistent to com . A_i is the set of copaths for hidden leaf node i . In Figure 4, if red node is hidden, then blue nodes are the copath in A_3 . S is the set of all the punctured seeds reconstructed from any A_i .

Now we define some bad events.

- $\text{Bad}_1 \Leftrightarrow$ there exists $e \in [d-1]$ and $i \in [2^e]$ such that $|S_{e,i}| \geq 2$.
- $\text{Bad}_2 \Leftrightarrow$ there exists $i \in [N]$ such that $|S_{d,i}| \geq 2\lambda/\log \lambda$.

Since the probability of querying a fixed commitment is $1/(2^\lambda - Q)$, we have $\Pr[|S_{d,i}| \geq c] \leq \Pr[X \geq c]$ where X follows $B(Q, 2/2^\lambda)$. Similar to (1), one have

$$\Pr[\text{Bad}_2] \leq \frac{4Q}{2^\lambda}$$

By Lemma 5 and (1),

$$\Pr[\text{Bad}_1 \vee \text{Bad}_2] \leq \frac{14Q}{2^\lambda} \quad (4)$$

In the following, we analyze the extracting condition without bad events.

- As S contains all possible (pdecom_I, I) , $\mathcal{A}$ wins the game only if $|S| > u$.
- By $\neg \text{Bad}_1$, we have

$$|S| \leq \sum_{i \in [N/2]} |S_{d,2i}| \cdot |A_{2i-1}| \leq \sum_{i \in [N/2]} |S_{d,2i}| \cdot |S_{d,2i-1}|.$$

Then, by $\neg \text{Bad}_2$, we have

$$|S| \leq 2N \left(\frac{\lambda}{\log \lambda} \right)^2.$$

Therefore, $\mathcal{A}$ cannot win the game without the bad events so we have

$$\text{Adv}_{\text{IC-VSC}}^{u\text{-esb}}(\mathcal{A}) \leq \Pr[\text{Bad}_1 \vee \text{Bad}_2] \leq \frac{14Q}{2^\lambda}$$

provided that $u = 2N \left(\frac{\lambda}{\log \lambda} \right)^2$. $\square$

Lemma 7 *Let $E : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ be an ideal cipher. Let $\mathcal{A}$ be an arbitrary adversary that makes Q queries to E . Then, $\mathcal{A}$'s multi-instance hiding advantage $\text{Adv}_{\text{IC-VSC}}^{Q_I\text{-mih}}(\mathcal{A})$ against IC-VSC is bounded by*

$$\text{Adv}_{\text{IC-VSC}}^{Q_I\text{-mih}}(\mathcal{A}) \leq \frac{(3 + 5N^2 + (2C + 8)^2 + \log N)Q_I}{2^\lambda} + \frac{(25 + 24 \log N)Q}{2^\lambda} + \frac{2}{2^\lambda}.$$

provided that $Q_I \leq \min \left\{ 2^{\frac{5(\lambda-24)}{6}}, \frac{2^{\lambda-1}}{(2C+4)N} \right\}$.

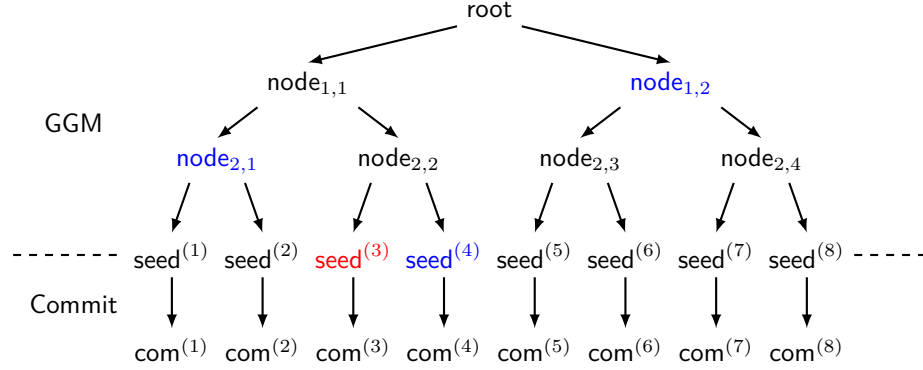


Fig. 4: The seed tree and commitments to the seeds. If the red seed is hidden, the blue nodes will be the copath of the red seed.

We basically bound advantage between the ideal world – hidden tapes are replaced by random strings – and the real world using the H-coefficient technique. After $m_{j,\bar{i}_j}$ are given, an attacker learns ideal cipher key, input, output relations from the tree construction, the commitment, and the tape generation. If these queries are inconsistent at this moment – either given instances themselves or with the attacker’s own ideal cipher queries – the attacker can distinguish the ideal world from the real world (i.e., the hidden messages are unchanged). Otherwise, the two worlds remain indistinguishable. The full proof in Supplementary Material B.2.

4 Application of VSC to BN++

4.1 Description of rBN++

In this section, we apply IC-VSC to BN++, dubbed *rBN++* (reduced BN++). rBN++ is a signature scheme containing three algorithms **KeyGen**, **Sign** and **Verify**, where **KeyGen** is assumed to be a one-way function. The signing and verification algorithms of the rBN++ can be found in the Algorithm 5 and Algorithm 6 in Supplementary material A. The major differences between the rBN++ and the original BN++ can be summarized into three points:

- The GGM trees are replaced by correlated GGM trees.
- The sizes of each commitments are reduced from 2λ bits to λ bits.
- Most of the random oracle calls are replaced by primitives based on an ideal cipher. Now, there are only 5 calls to random oracles for a signing query.

There are also some minor changes compared to the original BN++. The following modifications are made for either minor efficiency improvements or ease of proof.

- The message to be signed is hashed by $H_0 : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$ along with the public key.
- The salt is generated by running $H_3 : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$ with inputs the secret key $\mathbf{sk}$, hashed message μ , and internal randomness ρ , rather than randomly sampled.
- In each repetition, different salt salt_j is used. Each salt is generated from master salt salt by feeding into hash function $\text{ExpandSalt} : \{0, 1\}^{2\lambda} \rightarrow \{0, 1\}^{2\tau\lambda}$.
- The offsets $\Delta \mathbf{sk}_k, \Delta c_k, \Delta z_{k,j}$ are added to the last share, rather than to the first share.
- The function expanding h_1 and h_2 (the original function name in BN++ is Expand) is divided into two different functions: $\text{ExpandH1} : \{0, 1\}^{2\lambda} \rightarrow \mathbb{F}^{C \cdot \tau}$ and $\text{ExpandH2} : \{0, 1\}^{2\lambda} \rightarrow [N]^\tau$. These two functions are modeled as random oracles in the security proof.

4.2 Security Proof of rBN++

In this section, we prove the security of rBN++. The main difference between the security proofs for BN++ and rBN++ is that, semi-binding does not guarantee the uniqueness of H_1 -query input (μ, σ_1) . In the original BN++, individual commitment is of length 2λ bits so that the probability of H_1 -query input collision (not H_1 collision) is negligible. However, an EUF-KO adversary may find an H_1 -query input collision by querying to ideal cipher E , as the individual commitment size is less than 2λ bits. This case is covered in bad events Bad_4 , and Bad_5 in the security proof. Except this difference, the security can be proved similarly.

Theorem 1 (EUF-KO Security of rBN++). *Let $(N, \tau, \lambda, \mathbb{F})$ be parameters of rBN++ where $|\mathbb{F}| = 2^\lambda$ and $N = 2^d$. Assume that $H_1, H_2 : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$, ExpandH1 , and ExpandH2 are modeled as random oracles. Let $\mathcal{A}$ be an arbitrary adversary against the EUF-KO security of rBN++ that makes a total of Q random oracle queries and P ideal cipher queries. Then there exists PPT adversaries*

- $\mathcal{B}$ against OWF security of KeyGen with Q random oracle queries and P ideal cipher queries,
- $\mathcal{C}$ against extractable semi-binding security with P ideal cipher queries,

and $0 \leq \tau' < \tau$ such that

$$\begin{aligned} \text{Adv}_{\text{rBN}++}^{\text{euf-ko}} \leq & \frac{Q^2}{2^{2\lambda}} + \frac{(4\nu + 2)Q}{2^\lambda} + \frac{14P}{2^\lambda} + \text{Adv}_{\text{KeyGen}}^{\text{owf}}(\mathcal{B}) + \text{Adv}_{\text{IC-VSC}}^{u\text{-esb}}(\mathcal{C}) \\ & + Q_1 \cdot \sum_{i=\tau'}^{\tau} \binom{\tau}{i} \left(\frac{u}{|\mathbb{F}|} \right)^i \left(1 - \frac{u}{|\mathbb{F}|} \right)^{\tau-i} + \frac{Q_2}{N^{\tau-\tau'}} \end{aligned}$$

where $\nu = 2\lambda / \log \lambda$, $u = \nu^2 N / 2$, and $P, Q \leq 2^{\lambda-1}$.

Proof. Suppose that all the queries to H_1 , H_2 , E are listed in $\mathcal{Q}_1$, $\mathcal{Q}_2$, and $\mathcal{P}$, respectively. Let $|\mathcal{Q}_1| = Q_1$, $|\mathcal{Q}_2| = Q_2$, and $|\mathcal{P}| = P$. Suppose that **KeyGen** can be verified by C number of $\mathbb{F}$ -multiplications.

Algorithm 1. ExpandH1(h_1):

- 1 $h_1 \rightarrow \mathcal{H}_1$.
 - 2 $((\epsilon_{k,j})_{j \in [C]})_{k \in [\tau]} \leftarrow_{\$} (\mathbb{F}^C)^\tau$.
 - 3 Return $((\epsilon_{k,j})_{j \in [C]})_{k \in [\tau]}$.
-

Algorithm 2. ExpandH2(h_2):

- 1 $h_2 \rightarrow \mathcal{H}_2$.
 - 2 $(\bar{i}_1, \bar{i}_2, \dots, \bar{i}_\tau) \leftarrow_{\$} ([N])^\tau$.
 - 3 Return $(\bar{i}_1, \bar{i}_2, \dots, \bar{i}_\tau)$.
-

We program the random oracles for $\mathcal{A}$ as in Algorithm 1, 2, 3, and 4, respectively. For convenience, we denote $\Delta_k = (\Delta \mathbf{sk}_k, \Delta c_k, (\Delta z_{k,j})_{j \in [C]})$, $\alpha_k = (\alpha_{k,j})_{j \in [C]}$, and $\epsilon_k = (\epsilon_{k,j})_{j \in [C]}$. We manage sets $\mathcal{H}_1$ and $\mathcal{H}_2$ while the adversary is querying to identify any collision of H_1 and H_2 . We also manage sets $\text{Succ}_1[\text{salt}, h_1]$ (resp. $\text{Succ}_2[\text{salt}, h_2]$) for $\text{salt}, h_1 \in \{0, 1\}^{2\lambda}$ (resp. $\text{salt}, h_2 \in \{0, 1\}^{2\lambda}$). These sets contain $k \in [\tau]$ if and only if the k -th repetition passes the first (resp. second) multiplication check for a specific salt and first hash (resp. second hash). If one of such sets become equal to $[\tau]$, the adversary can forge a signature. Ext is the extractor in the u -ESB game.

$\text{MultCheck}_1^{(i)}$ and $\text{MultCheck}_2^{(i)}$ in Algorithm 4 are the first round and second round of the multiplication checking protocol for the i -th party, which is explicitly defined by

$$\begin{aligned} \text{MultCheck}_1^{(i)}((\epsilon_j)_j, m_i) &= (\epsilon_j \cdot x_j^{(i)} + a_j^{(i)})_j \\ \text{MultCheck}_2^{(i)}((\epsilon_j)_j, m_i, (\alpha_j^{(i)})_j) &= \sum_{j=1}^C \epsilon_j \cdot z_j^{(i)} - \sum_{j=1}^C \alpha_j \cdot y_j^{(i)} + c^{(i)} \end{aligned}$$

where $\mathbb{F}$ -elements $x_j^{(i)}, y_j^{(i)}, z_j^{(i)}, a_j^{(i)}$, and $c^{(i)}$ are components of m_i in specific positions, and $\mathbb{F}$ -vectors $(*_j)_j$ are of length C . We note that the length of m_i is $2C + 2$ as some outputs are fed into some inputs. **MultCheck** in Algorithm 3 is defined by combining above two functions as follows.

$$\text{MultCheck}((\epsilon_j)_j, m_i) = (\text{MultCheck}_1((\epsilon_j)_j, m_i), \text{MultCheck}_2((\epsilon_j)_j, m_i, (\alpha_j^{(i)})_j))$$

Algorithm 3. $H_1(\text{salt}, \sigma_1 = (\text{com}_k, \Delta_k)_{k \in [\tau]}):$

```

1  $h_1 \leftarrow_{\$} \{0, 1\}^{2\lambda}$ 
2 if  $h_1 \in \mathcal{H}_1$  then
3    $\perp$  Raise  $\text{Bad}_1$  and abort
4  $h_1 \rightarrow \mathcal{H}_1.$ 
5  $(\epsilon_1, \dots, \epsilon_\tau) \leftarrow \text{ExpandH1}(h_1)$ 
6  $(\text{salt}_1, \dots, \text{salt}_\tau) \leftarrow \text{ExpandSalt}(\text{salt})$ 
7  $\text{Succ}_1[\text{salt}, h_1] \leftarrow \emptyset$ 
8 for  $k \in [\tau]$  do
9   for  $(m_1, \dots, m_N) \in \text{Ext}((\text{salt}_k, k), \mathcal{P}, \text{com}_k)$  where  $\perp \notin (m_1, \dots, m_N)$ 
10    do
11       $m_N \leftarrow m_N \oplus \Delta_k$ 
12       $\text{sk} \leftarrow \text{msb}_\lambda \left( \sum_{i \in [N]} m_i \right)$ 
13      if  $\text{KeyGen}(\text{sk}) = \text{pk}$  then
14         $\perp$  Raise  $\text{Bad}_2$  and abort. // Secret key is found
15       $(\alpha^{(i)}, v^{(i)})_{i \in [N]} \leftarrow \text{MultCheck}(\epsilon_k, (m_i)_{i \in [N]})$ 
16      if  $\sum_{i \in [N]} v^{(i)} = 0$  then
17         $\perp$   $k \rightarrow \text{Succ}_1[\text{salt}, h_1]$  // Multiplication checking cheated
18 if  $|\text{Succ}_1[\text{salt}, h_1]| \geq \tau'$  then
19    $\perp$  Raise  $\text{Bad}_3$  and abort. // Too many cheated iterations
20  $(\text{salt}, \sigma_1, h_1) \rightarrow \mathcal{Q}_1$ 
21 Return  $h_1.$ 

```

where $(\alpha_j^{(i)})_j = \text{MultCheck}_1((\epsilon_j)_j, m_i)$. We note that the above programmings do not change the output distribution of random oracles.

We define bad events Bad consisting 6 small bad events Bad_i for $i = 1, \dots, 6$ (i.e., $\text{Bad} = \bigvee_{i=1}^6 \text{Bad}_i$). If the bad event happens, the game aborts and the adversary wins. Each small bad event is explained as follows.

- Bad_1 : A hash collision of H_1 or H_2 is found.
- Bad_2 : The PPT adversary $\mathcal{B}$ finds the preimage of KeyGen .
- Bad_3 : There are too many repetitions which are illegitimately passed in the first challenge.
- Bad_4 : In this case, the extracted message in fact passes the first challenge, but the message was not extracted at the time of querying to H_1 since the extractor lacks some query at that time. This event can be considered as preimage finding of the vector semi-commitment.
- Bad_5 : At least two punctured messages are extracted, where two punctured points are distinct. Since rBN++ signature contains a path of GGM tree rather than leaves, those two punctured messages can be combined to a

Algorithm 4. $H_2 \left(\text{salt}, h_1, \sigma_2 = \left((\alpha_k^{(i)}, v_k^{(i)})_{i \in [N], k \in [\tau]} \right) \right)$:

```

1   $h_1 \rightarrow \mathcal{H}_1$ 
2   $h_2 \leftarrow_{\$} \{0, 1\}^{2\lambda}$ 
3  if  $h_2 \in \mathcal{H}_2$  then
4     $\perp$  Raise  $\text{Bad}_1$  and abort.
5   $h_2 \rightarrow \mathcal{H}_2, (\text{salt}, h_1, \sigma_2, h_2) \rightarrow \mathcal{Q}_2$ .
6  if  $\exists k \in [\tau]$  such that  $\sum_{i \in [N]} v_k^{(i)} \neq 0$  then
7     $\perp$  Return  $h_2$ .
8  if  $\exists \sigma_1$  such that  $(h_1, \sigma_1) \in \mathcal{Q}_1$  then
9    Parse  $\sigma_1$  as  $(\text{com}_k, \Delta_k)_{k \in [\tau]}$ 
10    $(\epsilon_1, \dots, \epsilon_\tau) \leftarrow \text{ExpandH1}(h_1)$ 
11    $(\bar{i}_1, \dots, \bar{i}_\tau) \leftarrow \text{ExpandH2}(h_2)$ 
12 else
13    $\perp$  Return  $h_2$ .

14  $\text{Succ}_2[\text{salt}, h_2] \leftarrow \emptyset$ .
15 for  $k \in [\tau] \setminus \text{Succ}_1[\text{salt}, h_1]$  do
16    $\mathcal{J} \leftarrow \emptyset$ 
17   for  $(m_1, \dots, m_N) \in \text{Ext}(\text{salt}, \mathcal{P}, \text{com}_k)$  do
18     if  $m_N \neq \perp$  then
19        $\perp$   $m_N \leftarrow m_N \oplus \Delta_k$ 
20     if  $\perp \notin \{m_1, \dots, m_N\}$  then
21        $\text{sk} \leftarrow \text{msb}_\lambda \left( \sum_{i \in [N]} m_i \right)$ 
22       if  $\text{KeyGen}(\text{sk}) = \text{ct}$  then
23          $\perp$  Raise  $\text{Bad}_2$  and abort. // Secret key is found
24       for  $i \in [N]$ ,  $\beta^{(i)} \leftarrow \text{MultCheck}_1^{(i)}(\epsilon_k, m_i)$ 
25       for  $i \in [N]$ ,  $w^{(i)} \leftarrow \text{MultCheck}_2^{(i)}(\epsilon_k, m_i, \alpha_k)$ 
26       if  $\forall i \in [N]$ ,  $(\alpha_k^{(i)}, v_k^{(i)}) = (\beta^{(i)}, w^{(i)})$  then
27          $\perp$  Raise  $\text{Bad}_4$  and abort.
28       else if  $\exists j, \forall i \in [N] \setminus \{j\}$ ,  $(\alpha_k^{(i)}, v_k^{(i)}) = (\beta^{(i)}, w^{(i)})$  then
29          $\perp$   $j \rightarrow \mathcal{J}$ .

30   if  $|\mathcal{J}| \geq 2$  then
31      $\perp$  Raise  $\text{Bad}_5$  and abort.
32   else if  $\mathcal{J} = \{\bar{i}_k\}$  then
33      $\perp$   $k \rightarrow \text{Succ}_2[\text{salt}, h_2]$ 

34 if  $|\text{Succ}_2[\text{salt}, h_2]| \geq \tau - \tau'$  then
35    $\perp$  Raise  $\text{Bad}_6$  and abort. // Too many cheated iterations
36 Return  $h_2$ .
```

whole path for any undisclosed party. So, this event also means that the extractor finds a message which in fact passes the first challenge as Bad_4 .

- Bad_6 : there are too many repetitions which are illegitimately passed in the second challenge.

Bad_4 and Bad_5 did not happen when the “individual” commitment size is 2λ bits (i.e., vector commitment), as it is hard to find multiple preimages of individual commitment itself. On the other hand, in our case (i.e., vector semi-commitment), there may be a few preimages of a fixed commitment. Bad_4 describes an event that a directly found preimage from commitment queries passes the first challenge, and Bad_5 describes an event that multiple directly found punctured images from commitment queries passes the first challenge. We will show that the probability of such bad events are negligible in the following.

We have

$$\Pr[\mathcal{A} \text{ wins}] \leq \Pr[\text{Bad}] + \Pr[\mathcal{A} \text{ wins} \mid \neg \text{Bad}]$$

where the probability of each bad event is analyzed as follows.

- Upper bounding $\Pr[\text{Bad}_1]$. As $|h_1| = |h_2| = 2\lambda$, we have

$$\Pr[\text{Bad}_1] \leq \frac{Q^2}{2^{2\lambda}}.$$

Note that without Bad_1 , we can avoid the event for preimage/collision-finding of H_1 and H_2 .

- Upper bounding $\Pr[\text{Bad}_2]$. Let $\mathcal{B}$ be an OWF adversary that use $\mathcal{A}$ as a subroutine, and checks whether Bad_2 occurs by some root . Then, the winning probability of $\mathcal{B}$ is at least $\Pr[\text{Bad}_2]$, so we have

$$\Pr[\text{Bad}_2] \leq \text{Adv}_{\text{KeyGen}}^{\text{owf}}(\mathcal{B})$$

- Upper bounding $\Pr[\text{Bad}_3]$. By the definition of Ext , the number of extracted messages is always less or equal than u . Therefore, for each $k \in [\tau]$, the success probability of cheating the multiplication checking is upper bounded by $\frac{u}{|\mathbb{F}|}$ and we have

$$\Pr[\text{Bad}_3] \leq Q_1 \cdot \sum_{i=\tau'}^{\tau} \binom{\tau}{i} \left(\frac{u}{|\mathbb{F}|} \right)^i \left(1 - \frac{u}{|\mathbb{F}|} \right)^{\tau-i}$$

- Upper bounding $\Pr[\text{Bad}_4]$. Let us fix H_1 query, and assume that Bad_4 occurs at k -th iteration with $(m_1, \dots, m_N)$. Denote $m_{i,j} \in \mathbb{F}$ be the $(j+1)$ -th component of m_i for $j \in [2C+1]$, interpreting as $\mathbb{F}$ -vector. Since $(m_1, \dots, m_N)$ is extracted messages, there exists ideal cipher queries such that

$$E_{\text{seed}^{(i)}}(\text{ctr}[\langle k \rangle_B, i, 0]) = \text{com}_i \quad \text{for } i \in [N], \quad (\text{a})$$

$$E_{\text{seed}^{(i)}}(\text{ctr}[\langle k \rangle_B, i, j]) = m_{i,j} \quad \text{for } i \in [N], j \in [2C+1]. \quad (\text{b})$$

where $\text{seed}^{(i)}$ is the i -th seed. Let A and B be the list of ideal cipher queries of the form (a) and (b), and $A \cup B$ be the union list of A and B while the orders are preserved. Let $i^* \in [N]$ be the party index of the last query in A . As $k \notin \text{Succ}_1[\text{salt}, h_1]$, the above query is done after the query to H_1 . We divide two cases as follows.

- Case 1: The query $E_{\text{seed}^{(i^*)}}(\text{ctr}[\langle k \rangle_B, i^*, 0]) = \text{com}_{i^*}$ is the last query in $A \cup B$. Considering a fixed first challenge ϵ_k , the multiplication checking protocol is a set of linear relations of the shares. As all the other ideal cipher queries are fixed, there are at most 1 candidate of $\text{seed}^{(i^*)}$ which passes the multiplication checking protocol per H_1 query (i.e., all the other variables in the linear relations are fixed, but only $\text{seed}^{(i^*)}$ is not). Also, by (4) in Lemma 6, there are at most $\nu = \frac{2\lambda}{\log \lambda}$ of $(N-1)$ -tuples $(\text{seed}^{(i)})_{i \in [N] \setminus \{i^*\}}$ which satisfies (a) except with probability $14P/2^\lambda$. Since E should satisfy the equation for $\text{seed}^{(i^*)}$, we have

$$\Pr[\text{Bad}_4 \text{ with Case 1.}] \leq \frac{\nu Q_1}{2^\lambda - P} + \frac{14P}{2^\lambda} \leq \frac{2\nu Q_1}{2^\lambda} + \frac{14P}{2^\lambda}$$

provided that $P \leq 2^{\lambda-1}$.

- Case 2: The query $E_{\text{seed}^{(i^*)}}(\text{ctr}[\langle k \rangle_B, i^*, j]) = m_{i^*, j}$ is the last query in $A \cup B$. Similar to Case 1, the candidate of $m_{i^*, j}$ is now at most 1. The difference is that there are at most P candidates of $\text{seed}^{(i^*)}$, and E should satisfy both (a) and (b). So, there are at most νP candidates per H_1 query except $14P/2^\lambda$ probability by (4) in Lemma 6. We have

$$\Pr[\text{Bad}_4 \text{ with Case 2.}] \leq \frac{\nu Q_1 P}{(2^\lambda - P)^2} + \frac{14P}{2^\lambda} \leq \frac{2\nu Q_1}{2^\lambda} + \frac{14P}{2^\lambda}$$

provided that $P \leq 2^{\lambda-1}$.

All in all, we get

$$\Pr[\text{Bad}_4] \leq \frac{4\nu Q_1}{2^\lambda} + \frac{14P}{2^\lambda}$$

- Upper bounding $\Pr[\text{Bad}_5]$. Let us fix the query to H_2 , and assume that Bad_5 occurs by two extracted messages in the k -th iteration: $(m_1, \dots, m_N)$ adds j_1 to $\mathcal{J}$, and $(m'_1, \dots, m'_N)$ adds j_2 to $\mathcal{J}$. Then, by the definition of Ext , there exists $(m''_1, \dots, m''_N)$ in extracted message sets, such that $m''_i \in \{m_i, m'_i\}$ for $i \in [N] \setminus \{j_1, j_2\}$,¹ $m''_{j_1} = m'_{j_1}$ and $m''_{j_2} = m_{j_2}$. As $k \notin \text{Succ}_1[\text{salt}, h_1]$, $(m''_1, \dots, m''_N)$ induces Bad_4 , so

$$\Pr[\text{Bad}_5 \wedge \neg \text{Bad}_4] = 0.$$

Since the game immediately aborts if Bad_4 occurs, we can conclude that $\Pr[\text{Bad}_5] = 0$.

¹ We note that all the subtrees which do not contain both j_1 and j_2 should choose from the same kind of message. This is because of the structure of the GGM tree.

- Upper bounding $\Pr[\text{Bad}_6]$. If Bad_4 and Bad_5 do not happen, then $|\mathcal{J}| \leq 1$ for iterations in all queries to H_2 . So, we have

$$\Pr[\text{Bad}_6] \leq \frac{Q_2}{N^{\tau-\tau'}}.$$

All in all, we have

$$\begin{aligned} \Pr[\text{Bad}] &\leq \text{Adv}_{\text{KeyGen}}^{\text{owf}}(\mathcal{B}) + \frac{Q^2}{2^{2\lambda}} + \frac{4\nu Q_1}{2^\lambda} + \frac{14P}{2^\lambda} \\ &\quad + Q_1 \cdot \sum_{i=\tau'}^{\tau} \binom{\tau}{i} \left(\frac{u}{|\mathbb{F}|}\right)^i \left(1 - \frac{u}{|\mathbb{F}|}\right)^{\tau-i} + \frac{Q_2}{N^{\tau-\tau'}} \end{aligned} \quad (5)$$

What is left is to upper bound $\Pr[\mathcal{A} \text{ wins} \mid \neg \text{Bad}]$. Suppose that $\mathcal{A}$ outputs valid forgery

$$\left(\text{salt}, h_1, h_2, (\text{pdecom}_k, \Delta_k, \alpha_k^{(\bar{i}_k)})_{k \in [\tau]}\right)$$

but Bad does not occurs. Then, the forgery should satisfy followings.

- There exists $\sigma_1 = (\text{com}_k, \Delta_k)_{k \in [\tau]}$ such that $(\text{salt}, \sigma_1, h_1) \in \mathcal{Q}_1$.
- There exists $\sigma_2 = (\alpha_k^{(i)}, v_k^{(i)})_{i \in [N], k \in [\tau]}$ such that $(\text{salt}, h_1, \sigma_2, h_2) \in \mathcal{Q}_2$.
- Let $\text{Fail} = [\tau] \setminus (\text{Succ}_1[\text{salt}, h_1] \cup \text{Succ}_2[\text{salt}, h_1, h_2])$. Then, $\text{Fail} \neq \emptyset$.
- Fix $k^* \in \text{Fail}$, and let

$$\begin{aligned} \text{ExpandH1}(h_1) &= (\epsilon_k)_{k \in [\tau]}, \\ \text{ExpandH2}(h_2) &= (\bar{i}_k)_{k \in [\tau]}, \\ \text{Recon}(\text{salt}, \text{pdecom}_{k^*}, I_{k^*}) &= ((m_i)_{i \in I_{k^*}}, \text{com}) \end{aligned}$$

where $m_{\bar{i}_{k^*}} = \perp$ and $I_{k^*} = [N] \setminus \{\bar{i}_{k^*}\}$. Then, we have

$$\text{Verify}(\text{salt}, \text{com}, \text{pdecom}_{k^*}, I_{k^*}) = (m_i)_{i \in I_{k^*}}.$$

If $(m_1, \dots, m_N) \notin \text{Ext}(\text{com}, \Delta_{k^*})$, it means that there are more than u messages to be extracted (which is inhibited by the definition of the extractor). One can directly construct adversary $\mathcal{C}$ against semi-binding security of VSC using $\mathcal{A}$ as subroutine, so

$$\Pr[\mathcal{A} \text{ wins} \wedge (m_1, \dots, m_N) \notin \text{Ext}(\text{com}, \Delta_{k^*}) \mid \neg \text{Bad}] \leq \text{Adv}_{\text{IC-VSC}}^{u\text{-esb}}(\mathcal{C}). \quad (6)$$

Now assume that $(m_1, \dots, m_N) \in \text{Ext}(\text{com}, \Delta_{k^*})$. Since the bad events are only checked in H_1 and H_2 , the only chance to forgery is that additional ideal cipher queries after queries to H_1 and H_2 makes extra extracted messages to pass the challenges. It is equivalent to the event that the additional ideal cipher queries incur Bad_4 or Bad_5 . From the probability computation above, we have

$$\Pr[\mathcal{A} \text{ wins} \wedge (m_1, \dots, m_N) \in \text{Ext}(\text{com}, \Delta_{k^*}) \mid \neg \text{Bad}] \leq \frac{4\nu Q_1}{2^\lambda} + \frac{14P}{2^\lambda}. \quad (7)$$

We conclude the proof by combining 5, 6 and 7. $\square$

The EUF-CMA security is proved by game hopping technique as in the original BN++ scheme. Between the adjacent games, the security proof of BN++ mainly replaces a set of strings to random ones. Similarly, the security proof of rBN++ replaces strings in a straightforward manner, except one: commitment. Since the commitment size is λ bits, commitments cannot be replaced by random strings. This step is described in G_6 . Because of the page limit, we provide the sketch here and refer to Supplementary Material C for the full proof.

Theorem 2 (EUF-CMA Security of rBN++). *Assume that H_0 , H_1 , H_2 , ExpandH1, ExpandH2, and ExpandSalt are modeled as random oracles and that the (N, τ, λ) parameters of rBN++ are appropriately chosen where $|\mathbb{F}| = 2^\lambda$. For a PPT adversary $\mathcal{A}$ against the EUF-CMA security of rBN++ with a total of Q_{sig} signing oracle queries, Q random oracle queries, and P ideal cipher queries, there exist PPT adversaries*

- $\mathcal{B}$ against the EUF-KO security of rBN++,²
- $\mathcal{C}$ against the PRF security of H_3 ,³
- $\mathcal{D}$ against the multi-instance hiding security of IC-VSC

such that

$$\begin{aligned} \text{Adv}_{\text{rBN++}}^{\text{euf-cma}}(\mathcal{A}) &\leq \frac{(Q_{\text{sig}} + Q)^2}{2^{2\lambda}} + Q_{\text{sig}} \cdot \text{Adv}_{H_3}^{\text{prf}}(\mathcal{C}) \\ &\quad + \text{Adv}_{\text{IC-VSC}}^{\tau Q_{\text{sig}}\text{-mih}}(\mathcal{D}) + \text{Adv}_{\text{rBN++}}^{\text{euf-ko}}(\mathcal{B}), \end{aligned}$$

provided that $\tau \cdot Q_{\text{sig}} \leq \min \left\{ 2^{\frac{5(\lambda-24)}{6}}, \frac{2^{\lambda-1}}{(2C+4)N} \right\}$.

Proof (Sketch). Let $\mathcal{A}$ be an EUF-CMA adversary against rBN++ for given pk. Let G_0 be the original EUF-CMA game. Let $\mathcal{O}_{\text{sig}}$ be the signing oracle, and let Q_i for $i = 0, 1, 2$ be the number of queries made to H_i by $\mathcal{A}$. We only prove the security of the deterministic version of rBN++ ($\rho \leftarrow 0^n$) here. Without loss of generality, we assume that all messages in signing queries are distinct.

- G_1 : This game acts the same as G_0 except that it aborts if there exist two different queries on H_0 with the same outputs. As the output length of H_0 is 2λ , the abort probability is negligible.
- G_2 : $\mathcal{O}_{\text{sig}}$ replaces salt $\in \{0, 1\}^{2\lambda}$ with randomly sampled values, instead of computing $H_3(\text{sk}, \mu, \rho)$. As μ is always distinct for each query, the difference between this game and the previous one reduces to the PRF security of H_3 with secret key sk.
- G_3 : $\mathcal{O}_{\text{sig}}$ samples $h_1 \in \{0, 1\}^{2\lambda}$ at random instead of computing $H_1(\cdot)$, and programs the random oracle H_1 to output h_1 for the respective query. The simulation is aborted if the queries to H_1 have been made previously in a

² We assume $\mathcal{B}$ has the same amount of queries to random oracles.

³ H_3 itself is not a PRF, but it is used as a PRF with key prepending. We use this notation for convenience.

signing oracle query. As $\text{salt} \in \{0, 1\}^{2\lambda}$ is random, this game is indistinguishable from the previous game unless the simulation is aborted, and the probability of abort is negligible.

G_4 : $\mathcal{O}_{\text{sig}}$ now samples $h_2 \in \{0, 1\}^{2\lambda}$ at random instead of computing $H_2(\cdot)$, and also program the random oracle H_2 to output h_2 for the respective query. The simulation is aborted if the queries to H_2 have been made previously in a signing oracle query. As $h_1 \in \{0, 1\}^{2\lambda}$ is random, this game is indistinguishable with the previous game unless the simulation is aborted, and the probability of abort is negligible.

G_5 : $\mathcal{O}_{\text{sig}}$ now samples $(\text{salt}_1, \dots, \text{salt}_\tau) \in \{0, 1\}^{2\tau\lambda}$ at random instead of computing $\text{ExpandSalt}(\cdot)$, and also program the random oracle ExpandSalt to output $(\text{salt}_1, \dots, \text{salt}_\tau)$ for the respective query. The simulation is aborted if the queries to ExpandSalt have been made previously in a signing oracle query. As $\text{salt} \in \{0, 1\}^{2\lambda}$ is random, this game is indistinguishable with the previous game unless the simulation is aborted, and the probability of abort is negligible.

G_6 : For each $k \in [\tau]$, $\mathcal{O}_{\text{sig}}$ replaces the hidden tape $\text{tape}_k^{(\bar{i}_k)}$ by a random element in $\mathbb{F}^{2C+2}$. Since $(\text{decom}_i)_{i \in [\tau]}$ is computed independently from sk , the distribution of $(\text{pdecom}_i)_{i \in [\tau]}$ is also independent to sk .

The difference between G_5 and G_6 is bounded by the multi-instance hiding advantage of IC-VSC.

G_7 : $\mathcal{O}_{\text{sig}}$ replaces $(\Delta \text{sk}_k, \Delta c_k, (\Delta z_{k,j})_{j \in [C]})_{k \in [\tau]}$ with random elements instead of computing them using sk and S-box outputs. As the hidden tapes are random, the distribution of these variables does not change.

If the multiplication triple is wrong, then $v_k^{(\bar{i}_k)} \leftarrow -\sum_{i \neq \bar{i}_k} v_k^{(i)}$ is different from an honest value derived from legitimate calculation. However, $(\bar{i}_k)$ is unopened and the multiplication check is still passed. Since the signature oracle in G_7 does not depend on the secret key sk , it implies that G_7 can be reduced to the EUF-KO security.

This proves the theorem. □

4.3 Parameters and Efficiency

The parameters N and τ can be chosen to prevent the soundness attack [21]. As in the EUF-KO security proof, since the vector semi-commitment allows u valid partial decommitment information, the complexity of the soundness attack is slightly different from the original BN++ paper. The total complexity of the

attack $\mathcal{C}$ is computed as

$$P_1 = \sum_{k=\tau'}^{\tau} \binom{\tau}{k} \left(\frac{u}{|\mathbb{F}|} \right)^k \cdot \left(1 - \frac{u}{|\mathbb{F}|} \right)^{\tau-k}$$

$$P_2 = \frac{1}{N^{\tau-\tau'}}$$

$$\mathcal{C} = \min_{0 \leq \tau' \leq \tau} (1/P_1 + 1/P_2).$$

If the field size $|\mathbb{F}|$ is large enough, then τ' remains unchanged and it implies (N, τ) in rBN++ is same as the original BN++. For a small field, τ may be required to be much larger.

Efficiency improvements of rBN++ can be explained by the reduced number of random oracle calls and reduced signature size. In the literature, random oracles are implemented as a comparatively heavy hash functions such as SHAKE, whereas pseudorandom generators and ideal ciphers are implemented using the AES block cipher. In rBN++, the random oracle calls for commitments are translated into ideal cipher calls, and PRG calls for GGM trees are translated into a halved number of ideal cipher calls. Signature size is reduced by $\tau\lambda$ bits due to reduced commitment size. The results are summarized in Table 1. In this table, it is assumed that the secret key size is λ bits.

Scheme	Field Size	N	τ	RO call	PRG or IC call	Sig. size (B)
BN++	2^{128}	16	33	532	$1056C + 1518$	$1056C + 3792$
	2^{128}	256	17	4356	$8704C + 13022$	$544C + 3088$
rBN++	2^{128}	16	33	5	$1056C + 2079$	$1056C + 3264$
	2^{128}	256	17	5	$8704C + 17391$	$544C + 2816$

Table 1: Parameter sets for 128 bit security and the number of calls to the random oracles and the ideal cipher. Repeated multiplier is not applied for both schemes.

5 Performance

As many MPCitH-based signature schemes (specifically the first round candidates in the NIST call for additional post-quantum signatures) have similar forms with BN++, our improvements can be applied (possibly with some tweaks). However, our improvements may not be always applied in the best way for efficiency; if the probability of passing the first challenge is tight enough before applying our improvements (e.g., SDitH-L1-hyp [29] has the probability $2^{-71.2}$), the application involves increasing τ for security. We choose AIMer v2.0 [27] for

the performance measurement, as it is the best scheme for efficiency improvement. We will call it *reduced AImEr* (**rAImEr**). For application to similar non-interactive zero-knowledge proofs such as Threshold-Computation-in-the-Head (TCitH) [15] and VOLE-in-the-Head (VOLEitH) [4], we leave it as future work.

Environment. We developed **rAImEr** in C, with AVX2 instructions. A significant portion of our implementation is based on the AImEr v2.0 source code.⁴ For other schemes, we used packages that have been officially submitted to NIST PQC standardization project, and all were compiled using the default options in compilation script. Our experiments were measured in AMD Ryzen Threadripper PRO 5995WX 64-cores with 128GB memory. For a fair comparison, we measure the execution time for each signature scheme on the same CPU using the `taskset` command.

Scheme	$ pk $ (B)	$ sig $ (B)	Sign (Kc)	Verify (Kc)
Dilithium2 [28]	1,312	2,420	162	57
SPHINCS ⁺ -128f* [18]	32	17,088	38,216	2,158
SPHINCS ⁺ -128s* [18]	32	7,856	748,053	799
SDitH-Hypercube-gf256 [29]	132	8,496	20,820	10,935
FAEST-128f [3]	32	6,336	2,387	2,344
FAEST-128s [3]	32	5,006	20,926	20,936
AImEr-v2.0-128f [27]	32	5,888	788	752
AImEr-v2.0-128s [27]	32	4,160	5,926	5,812
rAImEr -128f	32	5,376	421	395
rAImEr -128s	32	3,904	2,826	2,730

*: -SHAKE256-simple

Table 2: Performance comparison of post-quantum signature schemes.

In Table 2, we compare the performance of **rAImEr** with various post-quantum signature schemes. We measured all the benchmarks of listed schemes in the same environment, and the table only contains publicly available implementations. It lists different schemes along with their respective public key sizes ($|pk|$), signature sizes ($|sig|$), signing times (Sign), and verification times (Verify). The sizes are provided in bytes (B), while the times for signing and verification are given in kilo-cycles (Kc). The table includes NIST-selected schemes Dilithium2, and SPHINCS⁺, as well as the first round MPCitH-based candidates of the NIST call for additional signatures like SDitH, FAEST, and AImEr.

From the data, we observe significant improvements in **rAImEr** compared to AImEr v2.0. **rAImEr** enjoys up to 109% faster signing and 112% faster verification,

⁴ <https://aimer-signature.org/>

as well as up to 8.7% smaller signature sizes. Compared to other MPCitH-based signature schemes, **rAIMer** enjoys the fastest signing and verification times and smallest signature sizes. When compared with selected algorithms, **rAIMer** shows significant superiority in performance compared to SPHINCS⁺, while it is quite inefficient compared to Dilithium.

Additionally, **rAIMer** can be combined with the proof-of-work technique proposed in [2]. With this technique, the signature size of 128-bit secure **rAIMer** becomes 5.1 KB (fast) / 3.7 KB (small) without a significant performance degradation. See Supplementary Material D for the details.

References

1. Aguilar-Melchor, C., Gama, N., Howe, J., Hülsing, A., Joseph, D., Yue, D.: The return of the SDitH. In: EUROCRYPT 2023. pp. 564–596. Springer (2023)
2. Baum, C., Beullens, W., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: One Tree to Rule Them All: Optimizing GGM Trees and OWFs for Post-Quantum Signatures. Cryptology ePrint Archive, Paper 2024/490 (2024), <https://eprint.iacr.org/2024/490>
3. Baum, C., Braun, L., de Saint Guilhem, C.D., Klooß, M., Majenz, C., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: FAEST. Technical report, National Institute of Standards and Technology, 2023 (2023), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>
4. Baum, C., Braun, L., de Saint Guilhem, C.D., Klooß, M., Orsini, E., Roy, L., Scholl, P.: Publicly Verifiable Zero-Knowledge and Post-Quantum Signatures from VOLE-in-the-Head. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023. pp. 581–615. Springer Nature Switzerland, Cham (2023)
5. Baum, C., Nof, A.: Concretely-Efficient Zero-Knowledge Arguments for Arithmetic Circuits and Their Application to Lattice-Based Cryptography. In: PKC 2020. pp. 495–526. Springer (2020)
6. Baum, C., Saint Guilhem, C.D.d., Kales, D., Orsini, E., Scholl, P., Zaverucha, G.: Banquet: Short and fast signatures from AES. In: PKC 2021. pp. 266–297. Springer (2021)
7. Beullens, W.: Sigma protocols for mq, pkp and sis, and fishy signature schemes. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020. pp. 183–211. Springer International Publishing, Cham (2020)
8. Bui, D., Carozza, E., Couteau, G., Goudarzi, D., Joux, A.: Short Signatures from Regular Syndrome Decoding, Revisited. Cryptology ePrint Archive, Paper 2024/252 (2024), <https://eprint.iacr.org/2024/252>
9. Bui, D., Cong, K., de Saint Guilhem, C.D.: Improved all-but-one vector commitment with applications to post-quantum signatures. Cryptology ePrint Archive, Paper 2024/097 (2024), <https://eprint.iacr.org/2024/097>
10. Cui, H., Liu, H., Yan, D., Yang, K., Yu, Y., Zhang, K.: ResolveD: Shorter signatures from regular syndrome decoding and vole-in-the-head. In: PKC 2024. pp. 229–258. Springer (2024)
11. Dobraunig, C., Kales, D., Rechberger, C., Schafneggger, M., Zaverucha, G.: Shorter Signatures Based on Tailor-Made Minimalist Symmetric-Key Crypto. In: ACM CCS 2022. pp. 843–857. Association of Computing Machinery

- (November 2022), <https://www.microsoft.com/en-us/research/publication/shorter-signatures-based-on-tailor-made-minimalist-symmetric-key-crypto/>
12. Don, J., Fehr, S., Majenz, C.: The Measure-and-Reprogram Technique 2.0: Multi-Round Fiat-Shamir and More. In: CRYPTO 2020. p. 602–631. Springer (2020)
 13. Feneuil, T.: Building MPCitH-based Signatures from MQ, MinRank, Rank SD and PKP. Cryptology ePrint Archive (2022)
 14. Feneuil, T., Joux, A., Rivain, M.: Syndrome Decoding in the Head: Shorter Signatures from Zero-Knowledge Proofs. In: Dodis, Y., Shrimpton, T. (eds.) CRYPTO 2022. pp. 541–572. Springer Nature Switzerland, Cham (2022)
 15. Feneuil, T., Rivain, M.: Threshold Computation in the Head: Improved Framework for Post-Quantum Signatures and Zero-Knowledge Arguments. Cryptology ePrint Archive, Paper 2023/1573 (2023), <https://eprint.iacr.org/2023/1573>
 16. Goldreich, O., Goldwasser, S., Micali, S.: How to Construct Random Functions. J. ACM **33**(4), 792–807 (aug 1986). <https://doi.org/10.1145/6490.6503>, <https://doi.org/10.1145/6490.6503>
 17. Guo, X., Yang, K., Wang, X., Zhang, W., Xie, X., Zhang, J., Liu, Z.: Half-tree: Halving the cost of tree expansion in cot and dpf. In: EUROCRYPT 2023. pp. 330–362. Springer (2023)
 18. Hulsing, A., Bernstein, D.J., Dobraunig, C., Eichlseder, M., Fluhrer, S., Gazdag, S.L., Kampanakis, P., Kolbl, S., Lange, T., Lauridsen, M.M., Mendel, F., Niederhagen, R., Rechberger, C., Rijneveld, J., Schwabe, P., Aumasson, J.P., Westerbaan, B., Beullens, W.: SPHINCS+. Technical report, National Institute of Standards and Technology, 2022 (2022), available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>
 19. Huth, J., Joux, A.: Mpc in the head using the subfield bilinear collision problem. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology – CRYPTO 2024. pp. 39–70. Springer Nature Switzerland, Cham (2024)
 20. Ishai, Y., Kushilevitz, E., Ostrovsky, R., Sahai, A.: Zero-knowledge from Secure Multiparty Computation. In: ACM STOC 2007. pp. 21–30 (2007)
 21. Kales, D., Zaverucha, G.: An Attack on Some Signature Schemes Constructed from Five-Pass Identification Schemes. In: Krenn, S., Shulman, H., Vaudenay, S. (eds.) Cryptology and Network Security. pp. 3–22. Springer International Publishing, Cham (2020)
 22. Kales, D., Zaverucha, G.: Efficient Lifting for Shorter Zero-Knowledge Proofs and Post-Quantum Signatures. Cryptology ePrint Archive, Paper 2022/588 (2022), <https://eprint.iacr.org/2022/588>
 23. Katz, J., Kolesnikov, V., Wang, X.: Improved Non-Interactive Zero Knowledge with Applications to Post-Quantum Signatures. In: ACM CCS 2018. pp. 525–537. ACM (2018)
 24. Kim, S., Ha, J., Son, M., Lee, B., Moon, D., Lee, J., Lee, S., Kwon, J., Cho, J., Yoon, H., Lee, J.: Aim: Symmetric primitive for shorter signatures with stronger security. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. p. 401–415. CCS ’23, Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3576915.3616579>, <https://doi.org/10.1145/3576915.3616579>
 25. Kosuge, H., Xagawa, K.: New Security Proofs of MPC-in-the-Head Signatures in the Quantum Random Oracle Model. Cryptology ePrint Archive, Paper 2025/1999 (2025), <https://eprint.iacr.org/2025/1999>
 26. Kosuge, H., Xagawa, K.: Private communication (September 2025)

27. Lee, J., Cho, J., Ha, J., Kim, S., Kwon, J., Lee, B., Lee, J., Lee, S., Moon, D., Son, M., Yoon, H.: The AIMer Signature Scheme (2024), version 2.0, available at https://kpmc.or.kr/competition_02.html
28. Lyubashevsky, V., Ducas, L., Kiltz, E., Lepoint, T., Schwabe, P., Seiler, G., Stehlé, D., Bai, S.: CRYSTALS-DILITHIUM. Technical report, National Institute of Standards and Technology, 2022 (2022), available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>
29. Melchor, C.A., Feneuil, T., Gama, N., Gueron, S., Howe, J., Joseph, D., Joux, A., Persichetti, E., Randrianarisoa, T.H., Rivain, M., Yue, D.: The Syndrome Decoding in the Head (SD-in-the-Head) Signature Scheme (2023), available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>
30. de Saint Guilhem, C.D., Orsini, E., Tanguy, T.: Limbo: Efficient Zero-Knowledge MPCitH-Based Arguments. In: ACM CCS 2021. p. 3022–3036. Association for Computing Machinery (2021)
31. Zaverucha, G., Chase, M., Derler, D., Goldfeder, S., Orlandi, C., Ramacher, S., Rechberger, C., Slamanig, D., Katz, J., Wang, X., Kolesnikov, V., Kales, D.: Picnic. Technical report, National Institute of Standards and Technology, 2020 (2022), available at <https://csrc.nist.gov/projects/post-quantum-cryptography/round-3-submissions>

Supplementary Material

A Description of the rBN++ Signature Scheme

Algorithm 5. $\text{Sign}(\text{sk}, \text{pk}, m)$ - reduced BN++, signing algorithm.

```

// Phase 1: Committing to the views of the parties.
1 Compute the hash of the message:  $\mu \leftarrow H_0(\text{pk}, m)$ 
2 Sample randomness:  $\rho \leftarrow_{\$} \{0, 1\}^\lambda$  ( $\rho \leftarrow 0^\lambda$  for deterministic signature)
3 Compute salt:  $\text{salt} \leftarrow H_3(\text{sk}, \mu, \rho)$ ,  $(\text{salt}_k)_{k \in [\tau]} \leftarrow \text{ExpandSalt}(\text{salt})$ .
4 for each repetition  $k \in [\tau]$  do
5   Sample a root seed:  $\text{root}_k \leftarrow_{\$} \{0, 1\}^\lambda$ .
6    $(\text{com}_k, \text{decom}_k, (\text{tape}_k^{(i)})_{i \in [N]}) \leftarrow \text{IC-VSC.Commit}(\text{salt}_k \| \langle k \rangle_B, \text{root}_k)$ .
7   For each party  $i$ , sample  $\text{sk}_k^{(i)} \leftarrow \text{Sample}(\text{tape}_k^{(i)})$ .
8    $\Delta \text{sk}_k \leftarrow \text{sk} - \text{root}_k, \text{sk}_k^{(N)} \leftarrow \text{sk}_k^{(N)} + \Delta \text{sk}_k$ .
9   for each gate  $g$  with index  $j$  do
10    if  $g$  is an addition with inputs  $(x_{k,j}, y_{k,j})$  then
11      For each party  $i$ , set the output share of  $z_{k,j}^{(i)} = x_{k,j}^{(i)} + y_{k,j}^{(i)}$ .
12    if  $g$  is a multiplication with inputs  $(x_{k,j}, y_{k,j})$  then
13      For each party  $i$ , sample an output share:
14         $z_{k,j}^{(i)} \leftarrow \text{Sample}(\text{tape}_k^{(i)})$ .
15      Compute output offset and adjust the last share:
16         $\Delta z_{k,j} = z_{k,j} - \sum_i z_{k,j}^{(i)}, z_{k,j}^{(N)} \leftarrow z_{k,j}^{(N)} + \Delta z_{k,j}$ .
17      For each party  $i$ , sample an helping values  $a$ :
18         $a_{k,j}^{(i)} \leftarrow \text{Sample}(\text{tape}_k^{(i)})$ .
19      Compute  $a_{k,j} = \sum_i a_{k,j}^{(i)}$  and set  $b_{k,j} = y_{k,j}$ .
20    Compute  $c_k = \sum_j a_{k,j} \cdot b_{k,j}$ .
21    For each party  $i$ , sample  $c_k^{(i)} \leftarrow \text{Sample}(\text{tape}_k^{(i)})$ .
22    Compute offset and adjust the last share :  $\Delta c_k = c_k - \sum_i c_k^{(i)}$ ,
23       $c_k^{(N)} \leftarrow c_k^{(N)} + \Delta c_k$ .
24 Set  $\sigma_1 \leftarrow (\text{salt}, (\text{com}_k, \Delta \text{sk}_k, \Delta c_k, (\Delta z_{k,j})_{j \in [C]})_{k \in [\tau]})$ .
// Phase 2: Challenging the checking protocol.
25 Compute challenge:  $h_1 \leftarrow H_1(\mu, \sigma_1)$ ,  $(\epsilon_{k,j})_{j \in [C]}_{k \in [\tau]} \leftarrow \text{ExpandH1}(h_1)$ .
// Phase 3: Committing to the checking protocol.
26 for each repetition  $k \in [\tau]$  do
27   Simulate the triple checking protocol as in Section 2.4 for all parties
28     with challenge  $\epsilon_{k,j}$ . The inputs are  $(x_{k,j}^{(i)}, y_{k,j}^{(i)}, z_{k,j}^{(i)}, a_{k,j}^{(i)}, b_{k,j}^{(i)}, c_k^{(i)})$ ,
29     and let  $\alpha_{k,j}^{(i)}$  and  $v_k^{(i)}$  be the broadcast values.
30 Set  $\sigma_2 \leftarrow (\text{salt}, ((\alpha_{k,j}^{(i)})_{j \in [C]}, v_k^{(i)})_{i \in [N]})_{k \in [\tau]}$ .
// Phase 4: Challenging the views of the MPC protocol.
31 Compute challenge hash:  $h_2 \leftarrow H_2(h_1, \sigma_2)$ ,  $(\bar{i}_k)_{k \in [\tau]} \leftarrow \text{ExpandH2}(h_2)$ .
// Phase 5: Opening the views.
32 for each repetition  $k \in [\tau]$  do
33    $\text{pdecom}_k \leftarrow \text{IC-VSC.Open}((\text{salt}, k), \text{decom}_k, [N] \setminus \{\bar{i}_k\})$ .
34 Output  $\sigma \leftarrow (\text{salt}, h_1, h_2, (\text{pdecom}_k, \Delta \text{sk}_k, \Delta c_k, (\Delta z_{k,j}, \alpha_{k,j}^{(\bar{i}_k)})_{j \in [C]})_{k \in [\tau]})$ .

```

Algorithm 6. Verify(pk, m, σ) - reduced BN++ signature scheme, verification algorithm.

```

1 Parse σ as (salt, h1, h2, (pdecomk, Δskk, Δck, (Δzk,j, αk,j(ik))j∈[C])k∈[τ]).
2 Compute the hash value of the message: μ ← H0(pk, m)
3 Expand hashes: (saltk)k∈[τ] ← ExpandSalt(salt),
   ((εk,j)j∈[C])k∈[τ] ← ExpandH1(h1), and (ik)k∈[τ] ← ExpandH2(h2).
4 for each repetition k ∈ [τ] do
5   ((tapek(i))i∈[N] \ {ik}, comk) ←
   IC-VSC.Recon(salt || ⟨τ⟩B, pdecomk, [N] \ {ik}).
6   for each party i ∈ [N] \ {ik} do
7     Sample skk(i) ← Sample(tapek(i)).
8     if i = N then
9       Adjust the last share: skk(i) ← skk(i) + Δskk
10    for each gate g with index j do
11      if g is an addition with inputs (xk,j, yk,j) then
12        Compute the output share of zk,j(i) = xk,j(i) + yk,j(i).
13      if g is a multiplication with inputs (xk,j, yk,j) then
14        Sample an output share: zk,j(i) ← Sample(tapek(i)).
15        if i = N then
16          Adjust the last share ze,j(i) ← ze,j(i) + Δze,j.
17        Sample ak,j(i) ← Sample(tapek(i)), and set bk,j(i) = yk,j(i).
18      Sample ck(i) ← Sample(tapek(i))
19      if i = N then
20        Adjust the last share ck(i) ← ck(i) + Δck.
21      Let pkk(i) be the final output shares.
22    Compute pkk(ik) = pk − ∑i≠ik pkk(i).
23 Set σ1 ← (salt, (comk, (pkk(i))i∈[N], Δskk, Δck, (Δzk,j)j∈[C])k∈[τ]).
24 Set h'1 ← H1(μ, σ1).
25 for each parallel execution k ∈ [τ] do
26   for each party i ∈ [N] \ {ik} do
27     Simulate the triple checking protocol as in Section 2.4 for all
     parties with challenge εk,j. The inputs are
     (xk,j(i), yk,j(i), zk,j(i), ak,j(i), bk,j(i), ck(i)), and let αk,j(i) and vk(i) be the
     broadcast values.
28   Compute vk(ik) = 0 − ∑i≠ik vk(i).
29 Set σ2 ← (salt, ((αk,j(i))j∈[C], vk(i))i∈[N])k∈[τ].
30 Set h'2 = H2(h'1, σ2).
31 Output Accept if h1 = h'1 and h2 = h'2.
32 Otherwise, output Reject.

```

B Proofs of Lemmas

B.1 Proofs for RO-VSC

Lemma 8 *Let $H_{\text{com}} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, $H_{\text{tree}} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ be random oracles. Let $\mathcal{A}$ be arbitrary adversary that makes Q_c queries to H_{com} and Q_t queries to H_{tree} . Then, the probability that $\mathcal{A}$ finds $\text{salt} \in \{0, 1\}^{2\lambda}$, $e, i \in \mathbb{N}$, and distinct $n, n' \in \{0, 1\}^\lambda$ such that*

$$\begin{cases} H_{\text{com}}(\text{salt}, i, H_{\text{tree}}(\text{salt}, e, i, n)) = H_{\text{com}}(\text{salt}, i, H_{\text{tree}}(\text{salt}, e, i, n')), \\ H_{\text{com}}(\text{salt}, i, (H_{\text{tree}}(\text{salt}, e, i, n) \oplus n)) = H_{\text{com}}(\text{salt}, i, (H_{\text{tree}}(\text{salt}, e, i, n') \oplus n')) \end{cases} \quad (8)$$

is at most $8Q/2^\lambda$.

Proof. Without loss of generality, assume that $\mathcal{A}$ queries to the random oracles with fixed salt, e , and i , and we omit the salt, e , and i input for each random oracle query. At the end of the game, we define $H_{\text{tree}}(x) = \perp$ (resp. $H_{\text{com}}(x) = \perp$) for non-queried input x to H_{tree} (resp. H_{com}), and we will consider that two $\perp$'s are not identical for simplicity. Let $H_{\text{tree}}(n) = l$, $n \oplus l = r$, $H_{\text{tree}}(n') = l'$, and $n' \oplus l' = r'$. We define

$$\begin{aligned} \mathcal{L}_1 &= \{(n, l, l') \in \{0, 1\}^{3\lambda} : H_{\text{tree}}(n) = l, H_{\text{com}}(l) = H_{\text{com}}(l'), l \neq l'\}, \\ \mathcal{L}_2 &= \{(n, r, r') \in \{0, 1\}^{3\lambda} : H_{\text{tree}}(n) \oplus n = r, H_{\text{com}}(r) = H_{\text{com}}(r'), r \neq r'\}, \\ \mathcal{L}_3 &= \{(n, l, l', r, r') \in \{0, 1\}^{5\lambda} : (n, l, l') \in \mathcal{L}_1, (n, r, r') \in \mathcal{L}_2\} \end{aligned}$$

and auxiliary events Aux_i for $i \in [3]$, where

$$\text{Aux}_i \Leftrightarrow |\mathcal{L}_i| > Q_c$$

and let $\text{Aux} = \text{Aux}_1 \vee \text{Aux}_2 \vee \text{Aux}_3$. Then, by Markov's inequality, we have

$$\begin{aligned} \Pr[\text{Aux}_1] &\leq \frac{\mathbb{E}[|\mathcal{L}_1|]}{Q_c} \leq \frac{1}{Q_c} \left(\frac{Q_t Q_c}{2^\lambda} + \frac{Q_t Q_c^2}{2^{2\lambda}} \right), \\ \Pr[\text{Aux}_2] &\leq \frac{\mathbb{E}[|\mathcal{L}_2|]}{Q_c} \leq \frac{1}{Q_c} \left(\frac{Q_t Q_c}{2^\lambda} + \frac{Q_t Q_c^2}{2^{2\lambda}} \right), \\ \Pr[\text{Aux}_3] &\leq \frac{\mathbb{E}[|\mathcal{L}_3|]}{Q_c} \leq \frac{1}{Q_c} \left(\frac{Q_t Q_c^2}{2^{2\lambda}} + \frac{2Q_t Q_c^3}{2^{3\lambda}} + \frac{Q_t Q_c^4}{2^{4\lambda}} \right). \end{aligned}$$

For each query $H_{\text{tree}}(n)$, we say **Bad** occurs if there exists $n' \neq n$ satisfies (8). Observing that **Bad** is the event of simultaneous two collisions, we divide **Bad** into left collisions $\mathcal{L}_i$ and right collisions $\mathcal{R}_i$, up to the freshness of l and r .

- $\mathcal{L}_1 \Leftrightarrow l = l'$ or l is freshly queried, and $H_{\text{com}}(l) = H_{\text{com}}(l')$. Then, n should satisfies

$$(H_{\text{tree}}(n) = l') \vee (H_{\text{tree}}(n) \neq l' \wedge H_{\text{com}}(l) = H_{\text{com}}(l')).$$

- $L_2 \Leftrightarrow l$ is not freshly queried, $l \neq l'$, and $H_{\text{com}}(l) = H_{\text{com}}(l')$. Then, n should satisfies

$$(n', l', H_{\text{tree}}(n)) \in \mathcal{L}_1.$$

- $R_1 \Leftrightarrow r = r'$ or r is freshly queried, and $H_{\text{com}}(r) = H_{\text{com}}(r')$. Then, n should satisfies

$$(H_{\text{tree}}(n) \oplus n = r') \vee (H_{\text{tree}}(n) \oplus n \neq r' \wedge H_{\text{com}}(r) = H_{\text{com}}(r')).$$

- $R_2 \Leftrightarrow r$ is not freshly queried, $r \neq r'$, and $H_{\text{com}}(r) = H_{\text{com}}(r')$. Then, n should satisfies

$$(n', r', H_{\text{tree}}(n) \oplus n) \in \mathcal{L}_2.$$

We have

$$\begin{aligned} \Pr[\text{Bad}] &\leq \Pr[\text{Aux}] + \sum_{i,j \in [2]} \Pr[L_i \wedge R_j \wedge \neg \text{Aux}] \\ &\leq \Pr[\text{Aux}] + \Pr[L_1 \wedge R_1] + \Pr[L_2 \wedge R_1 \wedge \neg \text{Aux}_1] \\ &\quad + \Pr[L_1 \wedge R_2 \wedge \neg \text{Aux}_2] + \Pr[L_2 \wedge R_2 \wedge \neg \text{Aux}_3] \end{aligned}$$

Each term can be bounded as follows.

- $\Pr[L_1 \wedge R_1]$: This can be divided into 4 sub-cases; L_1 has 2 cases, and R_1 has 2 cases. The first case – $l = l'$ and $r = r'$ at the same time – cannot achieved since it implies $n = n'$. Since the probability of each sub-cases is upper bounded by $Q_t^2/2^{2\lambda}$, this case is upper bounded by $3Q_t^2/2^{2\lambda}$.
- $\Pr[L_2 \wedge R_1 \wedge \neg \text{Aux}_1]$: Since L_2 and $r = r'$ cannot happen at the same time, this probability is upper bounded by $Q_t Q_c/2^{2\lambda}$.
- $\Pr[L_1 \wedge R_2 \wedge \neg \text{Aux}_2]$: Similarly as the previous case, this is upper bounded by $Q_t Q_c/2^{2\lambda}$.
- $\Pr[L_2 \wedge R_2 \wedge \neg \text{Aux}_3]$: $L_2 \wedge R_2$ means that $(n, l, l', r, r') \in \mathcal{L}_3$. So, this probability is upper bounded by $Q_c/2^\lambda$.

Then, we have

$$\Pr[\text{Bad}] \leq \frac{2Q_t + Q_c}{2^\lambda} + \frac{3Q_t^2 + 5Q_t Q_c}{2^{2\lambda}} + \frac{2Q_t Q_c^2}{2^{3\lambda}} + \frac{Q_t Q_c^3}{2^{4\lambda}} \leq \frac{8Q}{2^\lambda}$$

provided that $Q_c + Q_t \leq Q \leq 2^{\lambda-1}$, which concludes the proof. $\square$

Lemma 3 Let $H_{\text{com}} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ and $H_{\text{tree}} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ be random oracles. Let $\mathcal{A}$ be an arbitrary adversary that makes Q queries to the random oracles. Then $\mathcal{A}$'s u -extractable semi-binding advantage $\text{Adv}_{\text{RO-VSC}}^{u\text{-esb}}(\mathcal{A})$ against RO-VSC is bounded by

$$\text{Adv}_{\text{RO-VSC}}^{u\text{-esb}}(\mathcal{A}) \leq \frac{10Q}{2^\lambda},$$

for $u = 2N \left(\frac{\lambda}{\log \lambda} \right)^2$.

Proof. Intuitively, according to Lemma 8, the probability of finding a collision in commitments derived by a non-leaf node is negligible. Furthermore, for each leaf node, the number of multi-collisions is bounded by Chernoff bound. We now proceed to formally bound the adversary's advantage.

Let Q_t be the number of queries to H_{tree} and Q_c be the number of queries to H_{com} . Without loss of generality, assume that $\mathcal{A}$ queries to random oracles with fixed salt, and we omit the salt input for each random oracle query. Let $\mathcal{Q}_t$ and $\mathcal{Q}_c$ be the collection of queries to H_{tree} and H_{com} , respectively. At the end of the game, we define $H_{\text{tree}}(x) = \perp$ (resp. $H_{\text{com}}(x) = \perp$) for non-queried input x to H_{tree} (resp. H_{com}), and we consider that two $\perp$'s are not identical for simplicity.

We first define the extractor $\text{Ext}(\mathcal{Q}_t, \mathcal{Q}_c, \text{com} = (\text{com}_1, \dots, \text{com}_N))$ as follows.

1. For each $i \in [N]$, find $S_{d,i}$ such that

$$S_{d,i} = \{s : H_{\text{com}}(i, s) = \text{com}_i\}$$

2. For each $e \in [d-1]$ and $i \in [2^e]$, find $S_{e,i}$ such that

$$S_{e,i} = \{s : H_{\text{tree}}(e+1, i, s) \in S_{e+1,2i-1}, H_{\text{tree}}(e+1, i, s) \oplus s \in S_{e+1,2i}\}$$

3. For $i \in [N]$, let

$$A_i = \left\{ (p_1, \dots, p_d) : p_e \in S_{e,i_e} \text{ for } e \in [d] \right\}$$

where $i_e = (\lfloor (\bar{i} - 1)/2^{d-e} \rfloor \oplus 1) + 1$ for $e \in [d]$

4. Let S be the set of messages, where

$$S = \left\{ (s_1, \dots, s_N) : s_i = \perp \text{ if } S_{d,i} = \emptyset \text{ and } s_i \in S_{d,i} \text{ otherwise,} \right. \\ \left. \text{Recon}(p_1, \dots, p_d, \text{com}_i, I = [N] \setminus \{i\}) = (s_i)_{i \in I} \text{ for } (p_1, \dots, p_d) \in A_i \right\}$$

5. Finally, Ext outputs arbitrary u or less elements in S .

We define some bad events.

- $\text{Bad}_1 \Leftrightarrow$ there exists $e \in [d-1]$ and $i \in [2^e]$ such that $|S_{e,i}| \geq 2$.
- $\text{Bad}_2 \Leftrightarrow$ there exists $i \in [N]$ such that $|S_{d,i}| \geq 2\lambda/\log \lambda$.

By Lemma 8 and (1),

$$\Pr[\text{Bad}_1 \vee \text{Bad}_2] \leq \frac{10Q}{2^\lambda} \quad (9)$$

In the following, we analyze the extracting condition without bad events.

- As S contains all possible (pdecom_I, I) , $\mathcal{A}$ wins the game only if $|S| \geq u$.
- By $\neg \text{Bad}_1$, we have $|S| \leq \sum_{i \in [N/2]} |S_{d,2i}| \cdot |S_{d,2i-1}|$. Then, by $\neg \text{Bad}_2$, we have

$$|S| \leq 2N \left(\frac{\lambda}{\log \lambda} \right)^2.$$

Therefore, $\mathcal{A}$ cannot win the game without bad events so we have

$$\mathbf{Adv}_{\text{RO-VSC}}^{u\text{-esb}}(\mathcal{A}) \leq \Pr[\text{Bad}_1 \vee \text{Bad}_2] \leq \frac{10Q}{2^\lambda}$$

provided that $u = 2N \left(\frac{\lambda}{\log \lambda} \right)^2$. $\square$

Lemma 4 *Let $H_{\text{com}} : \{0,1\}^* \rightarrow \{0,1\}^\lambda$, $H_{\text{tree}} : \{0,1\}^* \rightarrow \{0,1\}^\lambda$, and $H_{\text{exp}} : \{0,1\}^* \rightarrow \{0,1\}^*$ be random oracles. Let $\mathcal{A}$ be an arbitrary adversary that makes Q queries to the random oracles. Then $\mathcal{A}$'s multi-instance hiding advantage $\mathbf{Adv}_{\text{RO-VSC}}^{Q_I\text{-mih}}(\mathcal{A})$ against RO-VSC is bounded by*

$$\mathbf{Adv}_{\text{RO-VSC}}^{Q_I\text{-mih}}(\mathcal{A}) \leq \frac{Q_I^2}{2^{2\lambda}} + \frac{Q}{2^\lambda}.$$

Proof. Let Q_t be the number of queries to H_{tree} , Q_c be the number of queries to H_{com} , and Q_e be the number of queries to H_{exp} . Let Q_I be the number of instances.

We will bound the advantage using the H-coefficient technique. Denote $\mathcal{I}$ as the ideal world where the hidden nodes are always replaced with random strings, and denote $\mathcal{R}$ the real world where the hidden nodes are remain unchanged. Let γ be the transcript of $\mathcal{A}$ which contains queries to the random oracles and the instances given in the game. The parent node of the hidden seed $\overline{\text{node}}$ is derived by $\text{node} = \text{seed}_{\bar{i}} \oplus \text{seed}_{((\bar{i}-1) \oplus 1)+1}$.

At the end of the game, root is given to the adversary complimentarily for the convenience of probability computation. Since it is given after whole game, the adversary cannot additionally query to all the oracles after root is given. After root is given, an adversary learns random oracle queries from the tree construction, commitment, and tape generation. If these queries are inconsistent – either internally or with the adversary's own queries – the adversary can distinguish the ideal world from the real world. Otherwise, the two worlds remain indistinguishable.

Now we define some events of bad transcripts as follows. These bad events describes the collision between adversary's random oracle queries and random oracle queries derived from instances.

- Bad_1 : two salt 's in the given instance collide. Since salt is sampled uniformly at random, $\Pr[\text{Bad}_1] \leq Q_I^2/2^{2\lambda}$.
- Bad_2 : a query $(\text{salt}, i, \text{seed}_{\bar{i}})$ is queried to H_{com} . $\Pr[\text{Bad}_2 \wedge \neg \text{Bad}_1] \leq Q_c/2^\lambda$.
- Bad_3 : a query $(\text{salt}, e, i, \overline{\text{node}})$ is queried to H_{tree} . $\Pr[\text{Bad}_3 \wedge \neg \text{Bad}_1] \leq Q_t/2^\lambda$.
- Bad_4 : a query $(\text{salt}, i, \text{seed}_i)$ is queried to H_{exp} . $\Pr[\text{Bad}_4 \wedge \neg \text{Bad}_1] \leq Q_e/2^\lambda$.

We say $\mathcal{T}_{\text{Bad}}$ be the set of bad transcripts, while $\mathcal{T}_{\text{Good}}$ be the complement of $\mathcal{T}_{\text{Bad}}$, and let T_{id} (resp. T_{re}) be the distribution of γ in $\mathcal{I}$ (resp. $\mathcal{R}$). As Q random oracle queries and Q_I instances of $m_{\bar{i}}$ are included in transcripts, for $\gamma \in \mathcal{T}_{\text{Good}}$,

$$\Pr[T_{\text{id}} = \gamma] = \Pr[T_{\text{re}} = \gamma] = \left(\frac{1}{2^\lambda} \right)^{Q+Q_I}.$$

So, the advantage is bounded by

$$\text{Adv}_{\text{RO-VSC}}^{Q_I\text{-mih}}(\mathcal{A}) \leq \frac{Q_I^2}{2^{2\lambda}} + \frac{Q}{2^\lambda}.$$

□

B.2 Proofs for IC-VSC

In this section, we provide the proofs of Lemma 5 and 7.

Lemma 5 *Let $E : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ be an ideal cipher and $\sigma : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ be an orthomorphism. Let $\mathcal{A}$ be an arbitrary adversary that makes Q queries to E . Then, the probability that $\mathcal{A}$ finds $\text{salt} = (\text{salt}_{\text{pt}}, \text{salt}_{\text{key}}, \mathbf{b}) \in \{0, 1\}^{\lambda+\lambda+8}$, $i \in \{0, 2, \dots, 254\}$, and distinct $n, n' \in \{0, 1\}^\lambda$ such that*

$$\begin{cases} E_{\sigma(n) \oplus E_{\text{salt}_{\text{key}}}(n \oplus \text{salt}_{\text{pt}})}(\text{salt}_{\text{pt}}[\lambda - 24] \parallel \mathbf{b} \parallel \langle i \rangle_B \parallel \langle 0 \rangle_B) \\ = E_{\sigma(n') \oplus E_{\text{salt}_{\text{key}}}(n' \oplus \text{salt}_{\text{pt}})}(\text{salt}_{\text{pt}}[\lambda - 24] \parallel \mathbf{b} \parallel \langle i \rangle_B \parallel \langle 0 \rangle_B), \\ E_{\sigma(n) \oplus E_{\text{salt}_{\text{key}}}(n \oplus \text{salt}_{\text{pt}}) \oplus n}(\text{salt}_{\text{pt}}[\lambda - 24] \parallel \mathbf{b} \parallel \langle i + 1 \rangle_B \parallel \langle 0 \rangle_B) \\ = E_{\sigma(n') \oplus E_{\text{salt}_{\text{key}}}(n' \oplus \text{salt}_{\text{pt}}) \oplus n'}(\text{salt}_{\text{pt}}[\lambda - 24] \parallel \mathbf{b} \parallel \langle i + 1 \rangle_B \parallel \langle 0 \rangle_B), \end{cases} \quad (3)$$

is at most $10Q/2^\lambda$.

Proof. At the end of game, we define $E_k(x) = \perp$ for non-queried input (k, x) to E , and we will consider that two $\perp$'s are not identical for simplicity. We also denote $\text{ctr}_l = \text{salt}_{\text{pt}}[\lambda - 24] \parallel \mathbf{b} \parallel \langle i \rangle_B \parallel \langle 0 \rangle_B$ and $\text{ctr}_r = \text{salt}_{\text{pt}}[\lambda - 24] \parallel \mathbf{b} \parallel \langle i + 1 \rangle_B \parallel \langle 0 \rangle_B$ for readability. Let $\sigma(n) \oplus E_{\text{salt}_{\text{key}}}(n \oplus \text{salt}_{\text{pt}}) = l$, $n \oplus l = r$, $\sigma(n') \oplus E_{\text{salt}_{\text{key}}}(n' \oplus \text{salt}_{\text{pt}}) = l'$, and $n' \oplus l' = r'$. We define

$$\begin{aligned} \mathcal{L}_1 &= \{(n, l, l') \in \{0, 1\}^{3\lambda} : \sigma(n) \oplus E_{\text{salt}_{\text{key}}}(n \oplus \text{salt}_{\text{pt}}) = l, \\ &\quad E_l(\text{ctr}_l) = E_{l'}(\text{ctr}_l), l \neq l'\}, \\ \mathcal{L}_2 &= \{(n, r, r') \in \{0, 1\}^{3\lambda} : \sigma(n) \oplus E_{\text{salt}_{\text{key}}}(n \oplus \text{salt}_{\text{pt}}) \oplus n = r, \\ &\quad E_r(\text{ctr}_r) = E_{r'}(\text{ctr}_r), r \neq r'\}, \\ \mathcal{L}_3 &= \{(n, l, l', r, r') \in \{0, 1\}^{5\lambda} : (n, l, l') \in \mathcal{L}_1, (n, r, r') \in \mathcal{L}_2\} \end{aligned}$$

and auxiliary events Aux_j for $j \in [3]$, where

$$\text{Aux}_j \Leftrightarrow |\mathcal{L}_j| > Q$$

and let $\text{Aux} = \text{Aux}_1 \vee \text{Aux}_2 \vee \text{Aux}_3$. Then, by Markov's inequality, we have

$$\begin{aligned} \Pr[\text{Aux}_1] &\leq \frac{\mathbb{E}[|\mathcal{L}_1|]}{Q} \leq \frac{1}{Q} \left(\frac{Q^2}{2^\lambda} + \frac{Q^3}{2^{2\lambda}} \right), \\ \Pr[\text{Aux}_2] &\leq \frac{\mathbb{E}[|\mathcal{L}_2|]}{Q} \leq \frac{1}{Q} \left(\frac{Q^2}{2^\lambda} + \frac{Q^3}{2^{2\lambda}} \right), \\ \Pr[\text{Aux}_3] &\leq \frac{\mathbb{E}[|\mathcal{L}_3|]}{Q} \leq \frac{1}{Q} \left(\frac{Q^3}{2^{2\lambda}} + \frac{2Q^4}{2^{3\lambda}} + \frac{Q^5}{2^{4\lambda}} \right). \end{aligned}$$

For each query $E_{\text{salt}_{\text{key}}}(n \oplus \text{salt}_{\text{pt}})$, we say **Bad** occurs if there exists $n' \neq n$ satisfies (3). Observing that **Bad** is the event of simultaneous two collisions, we divide **Bad** into left collisions $\mathbf{L}_i$ and right collisions $\mathbf{R}_i$, up to the freshness of l and r .

- $\mathbf{L}_1 \Leftrightarrow l = l'$ or l is freshly queried, and $E_l(l \oplus \text{salt}_{\text{pt}}) = E_{l'}(l' \oplus \text{salt}_{\text{pt}})$. Then, n should satisfies

$$(\sigma(n) \oplus E_{\text{salt}_{\text{key}}}(n \oplus \text{salt}_{\text{pt}}) = l') \vee \\ (\sigma(n) \oplus E_{\text{salt}_{\text{key}}}(n \oplus \text{salt}_{\text{pt}}) \neq l' \wedge E_l(\text{ctr}_l) = E_{l'}(\text{ctr}_l)).$$

- $\mathbf{L}_2 \Leftrightarrow l$ is not freshly queried, $l \neq l'$, and $E_l(l \oplus \text{salt}_{\text{pt}}) = E_{l'}(l' \oplus \text{salt}_{\text{pt}})$. Then, n should satisfies

$$(n', l', \sigma(n) \oplus E_{\text{salt}_{\text{key}}}(n \oplus \text{salt}_{\text{pt}})) \in \mathcal{L}_1.$$

- $\mathbf{R}_1 \Leftrightarrow r = r'$ or r is freshly queried, and $E_r(r \oplus \text{salt}_{\text{pt}}) = E_{r'}(r' \oplus \text{salt}_{\text{pt}})$. Then, n should satisfies

$$(\sigma(n) \oplus E_{\text{salt}_{\text{key}}}(n \oplus \text{salt}_{\text{pt}}) \oplus n = r') \vee \\ (\sigma(n) \oplus E_{\text{salt}_{\text{key}}}(n \oplus \text{salt}_{\text{pt}}) \oplus n \neq r' \wedge E_r(\text{ctr}_r) = E_{r'}(\text{ctr}_r)).$$

- $\mathbf{R}_2 \Leftrightarrow r$ is not freshly queried, $r \neq r'$, and $E_r(r \oplus \text{salt}_{\text{pt}}) = E_{r'}(r' \oplus \text{salt}_{\text{pt}})$. Then, n should satisfies

$$(n', r', (\sigma(n) \oplus E_{\text{salt}_{\text{key}}}(n \oplus \text{salt}_{\text{pt}}) \oplus n)) \in \mathcal{L}_2.$$

We have

$$\begin{aligned} \Pr[\mathbf{Bad}] &\leq \Pr[\mathbf{Aux}] + \sum_{i,j \in [2]} \Pr[\mathbf{L}_i \wedge \mathbf{R}_j \wedge \neg \mathbf{Aux}] \\ &\leq \Pr[\mathbf{Aux}] + \Pr[\mathbf{L}_1 \wedge \mathbf{R}_1] + \Pr[\mathbf{L}_2 \wedge \mathbf{R}_1 \wedge \neg \mathbf{Aux}_1] \\ &\quad + \Pr[\mathbf{L}_1 \wedge \mathbf{R}_2 \wedge \neg \mathbf{Aux}_2] + \Pr[\mathbf{L}_2 \wedge \mathbf{R}_2 \wedge \neg \mathbf{Aux}_3] \end{aligned}$$

Each term can be bounded as follows.

- $\Pr[\mathbf{L}_1 \wedge \mathbf{R}_1]$: This can be divided into 4 sub-cases; $\mathbf{L}_1$ has 2 cases, and $\mathbf{R}_1$ has 2 cases. The first case – $l = l'$ and $r = r'$ at the same time – cannot achieved since it implies $n = n'$. Since the probability of each sub-cases is upper bounded by $\binom{Q}{2}/(2^\lambda - Q)^2$, this case is upper bounded by $6Q^2/2^{2\lambda}$.
- $\Pr[\mathbf{L}_2 \wedge \mathbf{R}_1 \wedge \neg \mathbf{Aux}_1]$: Since $\mathbf{L}_2$ and $r = r'$ cannot happen at the same time, this probability is upper bounded by $Q^2/2^{2\lambda}$.
- $\Pr[\mathbf{L}_1 \wedge \mathbf{R}_2 \wedge \neg \mathbf{Aux}_2]$: Similarly as the previous case, this is upper bounded by $Q^2/2^{2\lambda}$.
- $\Pr[\mathbf{L}_2 \wedge \mathbf{R}_2 \wedge \neg \mathbf{Aux}_3]$: $\mathbf{L}_2 \wedge \mathbf{R}_2$ means that $(n, l, l', r, r') \in \mathcal{L}_3$. So, this probability is upper bounded by $Q/2^\lambda$.

Then, we have

$$\Pr[\text{Bad}] \leq \frac{3Q}{2^\lambda} + \frac{11Q^2}{2^{2\lambda}} + \frac{2Q^3}{2^{3\lambda}} + \frac{Q^4}{2^{4\lambda}} \leq \frac{10Q}{2^\lambda}$$

provided that $Q \leq 2^{\lambda-1}$, which concludes the proof. $\square$

Lemma 7 *Let $E : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ be an ideal cipher. Let $\mathcal{A}$ be an arbitrary adversary that makes Q queries to E . Then, $\mathcal{A}$'s multi-instance hiding advantage $\text{Adv}_{\text{IC-VSC}}^{Q_I\text{-mih}}(\mathcal{A})$ against IC-VSC is bounded by*

$$\text{Adv}_{\text{IC-VSC}}^{Q_I\text{-mih}}(\mathcal{A}) \leq \frac{(3 + 5N^2 + (2C + 8)^2 + \log N)Q_I}{2^\lambda} + \frac{(25 + 24 \log N)Q}{2^\lambda} + \frac{2}{2^\lambda}.$$

provided that $Q_I \leq \min \left\{ 2^{\frac{5(\lambda-24)}{6}}, \frac{2^{\lambda-1}}{(2C+4)N} \right\}$.

Proof. We will bound the advantage using the H-coefficient technique. Denote $\mathcal{I}$ the ideal world where the hidden tapes are always replaced to random strings, and denote $\mathcal{R}$ the real world where the hidden tapes are unchanged. Let $\mathcal{T}$ be the transcript of $\mathcal{A}$ which contains queries to the ideal cipher and the instances given in the game.

At the end of the game, `root` is given to the adversary complementarily for the convenience of probability computation. Since it is given after whole game, the adversary cannot additionally query to all the oracles after `root` is given. After `root` is given, an adversary learns ideal cipher key, input, output relations from the tree construction, commitment, and tape generation. If these queries are inconsistent – either internally or with the adversary's own queries – the adversary can distinguish the ideal from the real. Otherwise, the two worlds remain indistinguishable.

Define $\mathcal{Q}$ as the set of the ideal cipher query made by the adversary. Let $\mathcal{Q}_{\text{node}}$ (resp. $\mathcal{Q}_{\text{com}}, \mathcal{Q}_{\text{tape}}$) denote the set of the ideal cipher key-input-output relations learned from the tree construction (resp. commitment, tape generation). Specifically, $(K, X, Y) \in \mathcal{Q}_\bullet$ for $\bullet \in \{\text{node}, \text{com}, \text{tape}\}$ (or nothing) means that the query $E_K(X) = Y$ is got from $\bullet$. Additionally, define $\mathcal{Q}_{\text{ctr}}$ as $\mathcal{Q}_{\text{com}} \cup \mathcal{Q}_{\text{tape}}$. For each instance, parse `salt` = (`saltpt`, `saltkey`, `b`) and define `seed(i)` is the hidden seed computed after receiving `root`. And define `ance-1` for $e \in \{2, \dots, d\}$ as follows. If it is not confusing, we will omit subscript $j \in [Q_I]$ which is the instance index.

$$\begin{aligned} \text{anc}_d &= \text{seed}^{(\bar{i})} \\ \text{anc}_{e-1} &= \text{anc}_e \oplus \text{node}_{e, ((i_e-1) \oplus 1) + 1} \end{aligned}$$

where $i_e := \lfloor (\bar{i} - 1) / 2^{d-e} \rfloor + 1$. We note that $\text{node}_{e, ((i_e-1) \oplus 1) + 1}$ is the sibling node of node_{e, i_e} . In the real world, $\text{anc}_e = \text{node}_{e, i_e}$ represents the ancestor of $\text{node}_{d, \bar{i}}$ in the GGM tree at level $e \in [d]$. We give graphical description in Figure 5.

Let $\Delta = \text{seed}^{(\bar{i})} \oplus \text{node}_{d, \bar{i}}$ where $\text{node}_{d, \bar{i}}$ is the real hidden seed from the real GGM tree evaluation. By the correlation property, Δ remains the same across

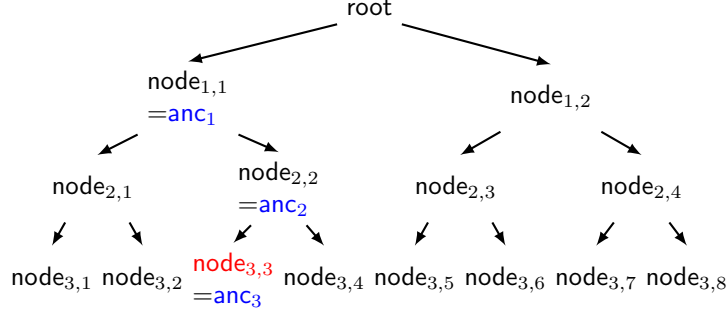


Fig. 5: The seed tree. If the red seed is hidden, $\text{anc}_1, \dots, \text{anc}_e$ will be the ancestor of hidden node.

all instance and $\text{anc}_e = \text{node}_{e,i_e} \oplus \Delta$ for all $e \in [d]$. In the real world, $\Delta = 0^\lambda$, whereas in the ideal world, Δ is chosen randomly from $\{0, 1\}^\lambda$.

For each instance and $e \in \{2, \dots, d\}$, the following queries are added to $\mathcal{Q}_{\text{node}}$.

- If i_e is odd (i.e., node_{e,i_e} is the left child), then the query

$$(\text{salt}_{\text{key}}, \text{anc}_{e-1} \oplus \text{salt}_{\text{pt}}, \text{anc}_e \oplus \sigma(\text{anc}_{e-1}))$$

is added to $\mathcal{Q}_{\text{node}}$.

- If i_e is even (i.e., node_{e,i_e} is the right child), then the query

$$(\text{salt}_{\text{key}}, \text{anc}_{e-1} \oplus \text{salt}_{\text{pt}}, \text{node}_{e,((i_e-1)\oplus 1)+1} \oplus \sigma(\text{anc}_{e-1}))$$

is added to $\mathcal{Q}_{\text{node}}$.

Since Δ is chosen uniformly at random, we remark that

$$\begin{aligned} \text{anc}_{e-1} \oplus \text{salt}_{\text{pt}} &= \Delta \oplus \text{node}_{e-1,i_{e-1}} \oplus \text{salt}_{\text{pt}} \\ \text{anc}_e \oplus \sigma(\text{anc}_{e-1}) &= \text{node}_{e,i_e} \oplus \text{node}_{e-1,i_{e-1}}, \\ &\quad \oplus (\text{node}_{e-1,i_{e-1}} \oplus \Delta) \oplus \sigma(\text{node}_{e-1,i_{e-1}} \oplus \Delta), \\ \text{node}_{e,i_e} \oplus \sigma(\text{anc}_{e-1}) &= \text{node}_{e,i_e} \oplus \sigma(\text{node}_{e-1,i_{e-1}} \oplus \Delta) \end{aligned}$$

for $e \in \{2, \dots, d\}$, which implies X, Y for $(K, X, Y) \in \mathcal{Q}_{\text{node}}$ are chosen uniformly at random up to the randomness of Δ .

For each instance, the following queries are added to $\mathcal{Q}_{\text{com}}$ and $\mathcal{Q}_{\text{tape}}$, respectively.

- An ideal cipher query from computing the hidden commitment:

$$(\text{seed}^{(\bar{i})}, \text{ctr}[\mathbf{b}, \bar{i}, 0], \text{com}_{\bar{i}}) \in \mathcal{Q}_{\text{com}}.$$

- Ideal cipher queries from computing the hidden tape, for $j \in [2C + 2]$:

$$(\text{seed}^{(\bar{i})}, \text{ctr}[\mathbf{b}, \bar{i}, j], m_{\bar{i}}[(j-1)\lambda + 1 : (j-1)\lambda + \lambda]) \in \mathcal{Q}_{\text{tape}}.$$

For each instance, $\log N$, 1 and $2C + 2$ queries are added to $\mathcal{Q}_{\text{node}}$, $\mathcal{Q}_{\text{com}}$ and $\mathcal{Q}_{\text{tape}}$, respectively. Therefore we have

$$|\mathcal{Q}_{\text{node}}| = \log N \cdot Q_I, \quad |\mathcal{Q}_{\text{com}}| = Q_I, \quad |\mathcal{Q}_{\text{tape}}| = (2C + 2)Q_I.$$

For $z \in \{0, 1\}^\lambda$, we define some subsets of $\mathcal{Q}_{\text{node}}$, $\mathcal{Q}_{\text{com}}$, $\mathcal{Q}_{\text{tape}}$ as follows.

$$\begin{aligned} \mathcal{Q}_{\text{node}}[z] &= \{(K, X, Y) \in \mathcal{Q}_{\text{node}} : K = z\}, \\ \mathcal{Q}_{\text{ctr}}^+[z] &= \{(K, X, Y) \in \mathcal{Q}_{\text{ctr}} : X = z\}, \\ \mathcal{Q}_{\text{com}}^-[z] &= \{(K, X, Y) \in \mathcal{Q}_{\text{com}} : Y = z\}. \end{aligned}$$

Now, we will define two kind of bad events; one is the restriction of (multi-)collisions (**Aux**), and the other is the real events (**Bad**). When one of the latter event happens, the adversary surely distinguishes both worlds. On the other hand, the former events are not critical to distinguishing itself but made for the proof easy. We define the former events as follows. We note that the probability bound of following events is computed assuming the ideal world since the probability of bad events in the ideal world should be bounded to use the H-coefficient technique.

- **Aux₁**: The maximal multi-collision in $\text{salt}_{\text{pt}}[:\lambda - 24]$ is greater than 12 or the maximal multi-collision in salt_{key} is greater than 12. From (2), the probability is bounded by

$$\Pr[\text{Aux}_1] \leq 2Q_I / (2^{16} \cdot 2^{2(\lambda-24)}) \leq Q_I / 2^{2\lambda-33}.$$

- **Aux₂**: There is a collision of at least two whole salts ($\text{salt}_{\text{pt}} \parallel \text{salt}_{\text{key}}$). The probability of this event is bounded by

$$\Pr[\text{Aux}_2] \leq Q_I^2 / 2^{2\lambda}.$$

- **Aux₃**: During computing nodes in $\text{IC-VSC.Commit}(\text{salt}_j, \text{root}_1)$, there exists an input collision on E . In other words, there exists $(\text{salt}_{\text{key}}, \text{node} \oplus \text{salt}_{\text{pt}})$ such that $E_{\text{salt}_{\text{key}}}(\text{node} \oplus \text{salt}_{\text{pt}})$ is queried at least twice. We remark that this event does not distinguishes the two worlds; as this event is not related to the replaced random strings, the probability of this event is same in the both worlds. This event may seems same with **Bad₅** in the later part, but **Bad₅** directly distinguishes these two worlds. The difference is:
 - **Aux₃** checks that if there is a key-input collision on E while computing a real GGM trees, regardless replaced random string. As computation of GGM trees are same on both the real world and the ideal world, this event does not distinguish the two worlds.
 - **Bad₅** checks that if there are a collision of queries made while computing the paths to the hidden nodes. As the hidden seeds will be replaced to a correlated random string, the collision will finds a contradiction on E computations.

As all the root nodes are same, the probability that $(\text{salt}_{\text{key}}, \text{root}_1 \oplus \text{salt}_{\text{pt}})$ collides with $\neg \text{Aux}_2$ is 0. Similar to a mathematical induction, we will see how distinct parent nodes make child nodes collide. Two distinct node evaluations $E_{\text{salt}_{\text{key}}}(\text{node} \oplus \text{salt}_{\text{pt}})$ and $E_{\text{salt}'_{\text{key}}}(\text{node}' \oplus \text{salt}'_{\text{pt}})$ from different instances incur an input collision at their child node levels if one of the two following pairs collide:

$$\begin{cases} \text{salt}_{\text{pt}} \oplus E_{\text{salt}_{\text{key}}}(\text{node} \oplus \text{salt}_{\text{pt}}) \oplus \sigma(\text{node}) \\ \text{salt}'_{\text{pt}} \oplus E_{\text{salt}'_{\text{key}}}(\text{node}' \oplus \text{salt}'_{\text{pt}}) \oplus \sigma(\text{node}') \end{cases} \quad (\text{left child})$$

$$\begin{cases} \text{salt}_{\text{pt}} \oplus E_{\text{salt}_{\text{key}}}(\text{node} \oplus \text{salt}_{\text{pt}}) \oplus \sigma(\text{node}) \oplus \text{node} \\ \text{salt}'_{\text{pt}} \oplus E_{\text{salt}'_{\text{key}}}(\text{node}' \oplus \text{salt}'_{\text{pt}}) \oplus \sigma(\text{node}') \oplus \text{node}' \end{cases} \quad (\text{right child})$$

where salt_{key} and $\text{salt}'_{\text{key}}$ also collide. If $\text{node} \oplus \text{salt}_{\text{pt}}$ and $\text{node}' \oplus \text{salt}'_{\text{pt}}$ collide, then those two pairs cannot collide since σ is an orthomorphism. Otherwise, each pair will collide with probability at most

$$\frac{1}{2^\lambda - (\# \text{ of previous evaluations with } \text{salt}_{\text{key}})} \leq \frac{1}{(2^\lambda - 24N)}$$

conditioned $\text{salt}_{\text{key}} = \text{salt}'_{\text{key}}$.

Then, with $\neg \text{Aux}_1$, the inputs collide at their child node levels with probability at most

$$\frac{2}{2^\lambda(2^\lambda - (\# \text{ of previous evaluations with } \text{salt}_{\text{key}}))} \leq \frac{2}{2^\lambda(2^\lambda - 24N)}.$$

Similarly, two nodes in a single instance collides with probability at most

$$\frac{2}{2^\lambda - 24N}.$$

By a hybrid argument, we have

$$\begin{aligned} \Pr[\text{Aux}_3 \wedge \neg \text{Aux}_1 \wedge \neg \text{Aux}_2] &\leq \binom{Q_I}{2} \binom{2N}{2} \cdot \frac{2}{2^\lambda(2^\lambda - 24N)} + \binom{2N}{2} \cdot \frac{2Q_I}{2^\lambda - 24N} \\ &\leq \frac{4N^2Q_I^2}{2^{2\lambda}} + \frac{4N^2Q_I}{2^\lambda} \leq \frac{5N^2Q_I}{2^\lambda} \end{aligned}$$

provided that $N \leq 2^{\lambda-1}$.

- Aux_4 : The maximal multi-collision in $\text{com}_{\bar{i}}$ is greater than 12. As there are at most

$$((2C + 2)N + (N - 1))Q_I \leq 2^{\lambda-1}.$$

queries to E made during Commit, on top of $\neg \text{Aux}_3$, the probability that $\text{com}_{\bar{i}} = x$ for some $x \in \{0, 1\}^\lambda$ is at most $2/2^\lambda$. From (2), we have

$$\Pr[\text{Aux}_4 \wedge \neg \text{Aux}_3] \leq Q_I / (2^{16} \cdot 2^{2\lambda-1}) \leq Q_I / 2^{2\lambda+15}.$$

Now we introduce the bad events. As a single incompatible ideal cipher query distinguishes the two worlds, each bad event corresponds to two of $\{\mathcal{Q}, \mathcal{Q}_{\text{ctr}}, \mathcal{Q}_{\text{node}}\}$ queries collide. As a collision between ideal cipher queries made by the adversary – both in $\mathcal{Q}$ – does not contribute to the distinguishing advantage, there are 5 bad events.

- **Bad₁**: There exists $(K, X, Y) \in \mathcal{Q}$ and $(K', X', Y') \in \mathcal{Q}_{\text{ctr}}$ such that $(K, X) = (K', X')$ or $(K, Y) = (K', Y')$. Since K' is randomly chosen and Y' is randomly chosen if it is from $\mathcal{Q}_{\text{tape}}$, we have

$$\begin{aligned} \Pr[(K, X) = (K', X')] &\leq \begin{cases} \frac{1}{2^\lambda} & \text{if } (K', X', Y') \in \mathcal{Q}_{\text{ctr}}^+[X], \\ 0 & \text{otherwise} \end{cases} \\ \Pr[(K, Y) = (K', Y')] &\leq \begin{cases} \frac{1}{2^\lambda} & \text{if } (K', X', Y') \in \mathcal{Q}_{\text{com}}^-[Y], \\ \frac{1}{2^{2\lambda}} & \text{if } (K', X', Y') \in \mathcal{Q}_{\text{tape}}, \\ 0 & \text{otherwise.} \end{cases} \end{aligned}$$

By assuming $\neg(\text{Aux}_1 \vee \text{Aux}_4)$, we get $|\mathcal{Q}_{\text{ctr}}^+[X]| \leq 12$, $|\mathcal{Q}_{\text{com}}^-[Y]| \leq 12$ and

$$\begin{aligned} \Pr[\text{Bad}_1 \wedge \neg(\text{Aux}_1 \vee \text{Aux}_4)] &\leq \sum_{X \in \{0,1\}^\lambda} \frac{|\mathcal{Q}_{\text{ctr}}^+[X]| \cdot (\# \text{ of } (\cdot, X, \cdot) \in \mathcal{Q})}{2^\lambda} \\ &\quad + \sum_{Y \in \{0,1\}^\lambda} \frac{|\mathcal{Q}_{\text{com}}^-[Y]| \cdot (\# \text{ of } (\cdot, \cdot, Y) \in \mathcal{Q})}{2^\lambda} \\ &\quad + \frac{|\mathcal{Q}_{\text{tape}}| \cdot Q}{2^{2\lambda}} \\ &\leq \frac{24Q}{2^\lambda} + \frac{(2C+2)Q_I Q}{2^{2\lambda}} \leq \frac{25Q}{2^\lambda}. \end{aligned}$$

- **Bad₂**: There exists $(K, X, Y) \in \mathcal{Q}$ and $(K, X', Y') \in \mathcal{Q}_{\text{node}}$ such that $X = X'$ or $Y = Y'$. Recall that random sampling of Δ implies random sampling of X' 's and Y' 's. For each query $(K, X, Y) \in \mathcal{Q}$ and $(K, X', Y') \in \mathcal{Q}_{\text{node}}[K]$, we have

$$\Pr[X = X' \vee Y = Y'] \leq \frac{2}{2^\lambda}.$$

By assuming $\neg \text{Aux}_1$, we get $|\mathcal{Q}_{\text{node}}[K]| \leq 12 \log N$ and

$$\Pr[\text{Bad}_2 \wedge \neg \text{Aux}_1] \leq \sum_{K \in \{0,1\}^\lambda} \frac{|\mathcal{Q}_{\text{node}}[K]| \cdot (\# \text{ of } (K, \cdot, \cdot) \in \mathcal{Q})}{2^{\lambda-1}} \leq \frac{24 \log N \cdot Q}{2^\lambda}.$$

- **Bad₃**: There exist distinct triples $(K, X, Y), (K', X', Y') \in \mathcal{Q}_{\text{ctr}}$ such that $(K, X) = (K', X')$ or $(K, Y) = (K', Y')$. If (K, X, Y) and (K', X', Y') are from the same instance, we trivially have $K = K'$, $X \neq X'$, and $\Pr[Y = Y'] = 1/2^\lambda$, so

$$\Pr[\text{Bad}_3 \text{ occurs in a same instance}] \leq \binom{2C+2}{2} \frac{Q_I}{2^\lambda}.$$

Now, assume that (K, X, Y) and (K', X', Y') are from different instance. There are three cases that Bad_3 occurs in different instances:

- two queries in $\mathcal{Q}_{\text{ctr}}^+[X]$ collide at key (i.e., $K = K'$);
- two queries in $\mathcal{Q}_{\text{com}}^-[Y]$ collide at key (i.e., $K = K'$);
- a query in $\mathcal{Q}_{\text{ctr}}$ and a query in $\mathcal{Q}_{\text{tape}}$ collide at key and output.

Since $\neg \text{Aux}_3$ prevent the input collision while computing nodes in IC-VSC.Commit , we have $\Pr[K = K' \wedge \neg \text{Aux}_3] \leq \frac{2}{2^\lambda}$ and the probability is bounded by

$$\begin{aligned} & \Pr[\text{Bad}_3 \text{ occurs in different instances} \wedge \neg(\text{Aux}_1 \vee \text{Aux}_3 \vee \text{Aux}_4)] \\ & \leq \frac{2}{2^\lambda} \left(\sum_{X \in \{0,1\}^\lambda} \binom{|\mathcal{Q}_{\text{ctr}}^+[X]|}{2} + \sum_{Y \in \{0,1\}^\lambda} \binom{|\mathcal{Q}_{\text{com}}^-[Y]|}{2} \right) + \frac{|\mathcal{Q}_{\text{ctr}}| \cdot |\mathcal{Q}_{\text{tape}}|}{2^{2\lambda}} \\ & \leq \frac{12(|\mathcal{Q}_{\text{ctr}}| + |\mathcal{Q}_{\text{com}}|)}{2^\lambda} + \frac{(2C+2)^2 Q_I^2}{2^{2\lambda}} \\ & \leq \frac{12(2C+4)Q_I}{2^\lambda} + \frac{(2C+2)^2 Q_I^2}{2^{2\lambda}}. \end{aligned}$$

All in all,

$$\begin{aligned} \Pr[\text{Bad}_3 \wedge \neg(\text{Aux}_1 \vee \text{Aux}_3 \vee \text{Aux}_4)] & \leq \frac{(2C+7)^2 Q_I}{2^\lambda} + \frac{(2C+2)^2 Q_I^2}{2^{2\lambda}} \\ & \leq \frac{(2C+7)^2 Q_I}{2^\lambda} + \frac{(2C+2)Q_I}{2^\lambda} \\ & \leq \frac{(2C+8)^2 Q_I}{2^\lambda}. \end{aligned}$$

- Bad_4 : There exist distinct triples $(K, X, Y) \in \mathcal{Q}_{\text{node}}$ and $(K', X', Y') \in \mathcal{Q}_{\text{ctr}}$ such that $(K, X) = (K', X')$ or $(K, Y) = (K', Y')$. Since

$$\begin{aligned} \Pr\left[\Delta \xleftarrow{\$} \{0,1\}^\lambda : X = X' \vee Y = Y'\right] & \leq \frac{2}{2^\lambda} \\ \Pr\left[\text{salt}_{\text{key}} \xleftarrow{\$} \{0,1\}^\lambda : K = K' \mid \Delta = \delta\right] & \leq \frac{1}{2^\lambda} \end{aligned}$$

for any fixed δ , we have

$$\Pr[\text{Bad}_4] \leq \frac{2|\mathcal{Q}_{\text{node}}| \cdot |\mathcal{Q}_{\text{ctr}}|}{2^{2\lambda}} \leq \frac{4(C+1) \log N \cdot Q_I^2}{2^{2\lambda}} \leq \frac{2Q_I}{2^\lambda}$$

- Bad_5 : There exist distinct triples $(K, X, Y), (K', X', Y') \in \mathcal{Q}_{\text{node}}$ such that $(K, X) = (K', X')$ or $(K, Y) = (K', Y')$. Recall that $\text{anc}_{e-1} = \text{node}_{e-1, i_{e-1}} \oplus \Delta$. Then, we have

$$\begin{aligned} & (\text{salt}_{\text{key}}, \text{anc}_{e-1} \oplus \text{salt}_{\text{pt}}) = (\text{salt}'_{\text{key}}, \text{anc}'_{e'-1} \oplus \text{salt}'_{\text{pt}}) \\ \Leftrightarrow & (\text{salt}_{\text{key}}, \text{anc}_{e-1} \oplus \Delta \oplus \text{salt}_{\text{pt}}) = (\text{salt}'_{\text{key}}, \text{anc}'_{e'-1} \oplus \Delta \oplus \text{salt}'_{\text{pt}}) \\ \Leftrightarrow & (\text{salt}_{\text{key}}, \text{node}_{e-1, i_{e-1}} \oplus \text{salt}_{\text{pt}}) = (\text{salt}'_{\text{key}}, \text{node}_{e'-1, i'_{e'-1}} \oplus \text{salt}'_{\text{pt}}) \end{aligned}$$

which implies

$$\Pr[(K, X) = (K', X') \wedge \neg \text{Aux}_3] = 0$$

since $\neg \text{Aux}_3$ restricts that there is no key-input collision while computing nodes in $\text{IC-VSC.Commit}()$. Similarly, we have

$$\begin{aligned} (\text{salt}_{\text{key}}, \text{anc}_e \oplus \sigma(\text{anc}_{e-1})) &= (\text{salt}'_{\text{key}}, \text{anc}'_{e'} \oplus \sigma(\text{anc}'_{e'-1})) \\ \Leftrightarrow (\text{salt}_{\text{key}}, \text{node}_{e,i_e} \oplus \sigma(\text{node}_{e-1,i_{e-1}})) &= (\text{salt}'_{\text{key}}, \text{node}_{e',i'_{e'}} \oplus \sigma(\text{node}_{e'-1,i'_{e'-1}})), \\ (\text{salt}_{\text{key}}, \text{anc}_e \oplus \sigma(\text{anc}_{e-1})) &= (\text{salt}'_{\text{key}}, \text{node}_{e',i'_{e'}} \oplus \sigma(\text{anc}'_{e'-1})) \\ \Leftrightarrow (\text{salt}_{\text{key}}, \text{node}_{e,i_e} \oplus \sigma(\text{node}_{e-1,i_{e-1}}) \oplus \Delta) &= (\text{salt}'_{\text{key}}, \text{node}_{e',i'_{e'}} \oplus \sigma(\text{node}_{e'-1,i'_{e'-1}})) \\ (\text{salt}_{\text{key}}, \text{node}_{e,i_e} \oplus \sigma(\text{anc}_{e-1})) &= (\text{salt}'_{\text{key}}, \text{node}_{e',i'_{e'}} \oplus \sigma(\text{anc}'_{e'-1})) \\ \Leftrightarrow (\text{salt}_{\text{key}}, \text{node}_{e,i_e} \oplus \sigma(\text{node}_{e-1,i_{e-1}})) &= (\text{salt}'_{\text{key}}, \text{node}_{e',i'_{e'}} \oplus \sigma(\text{node}_{e'-1,i'_{e'-1}})), \end{aligned}$$

where only the second proposition have the probabilistic variable Δ . So, there is no chance to satisfy the first and third proposition with $\neg \text{Aux}_3$. Then, it implies that

$$\Pr[(K, Y) = (K', Y') \wedge \neg \text{Aux}_3] \leq \frac{1}{2^{2\lambda}}.$$

All in all, we get

$$\Pr[\text{Bad}_5 \wedge \neg \text{Aux}_3] \leq \frac{(\log N \cdot Q_I)^2}{2^{2\lambda}} \leq \frac{\log N \cdot Q_I}{2^\lambda}$$

With $\text{Aux} = \text{Aux}_1 \vee \dots \vee \text{Aux}_4$ and $\text{Bad} = \text{Bad}_1 \vee \dots \vee \text{Bad}_5$, we have

$$\begin{aligned} \Pr[\text{Aux} \vee \text{Bad}] &\leq \frac{Q_I}{2^{2\lambda-33}} + \frac{Q_I^2}{2^{2\lambda}} + \frac{5N^2 Q_I}{2^\lambda} + \frac{Q_I}{2^{2\lambda+15}} \\ &\quad + \frac{25Q}{2^\lambda} + \frac{24 \log N \cdot Q}{2^\lambda} + \frac{(2C+8)^2 Q_I}{2^\lambda} + \frac{2Q_I}{2^\lambda} + \frac{\log N \cdot Q_I}{2^\lambda} \\ &\leq \frac{(3+5N^2+(2C+8)^2+\log N)Q_I}{2^\lambda} + \frac{(25+24 \log N)Q}{2^\lambda} + \frac{2}{2^\lambda} \end{aligned}$$

where the inequalities assume that $\lambda \geq 78$.

We say $\mathcal{T}_{\text{Bad}}$ be the set of bad transcripts, while $\mathcal{T}_{\text{Good}}$ be the complement of $\mathcal{T}_{\text{Bad}}$, and let T_{id} (resp. T_{re}) be the distribution of γ in $\mathcal{I}$ (resp. $\mathcal{R}$). As Q ideal cipher queries and Q_I instances of $m_{\bar{i}}$ are included in transcripts, for $\gamma \in \mathcal{T}_{\text{Good}}$,

$$\Pr[\gamma = T_{\text{id}}] = \frac{1}{(2^{(2C+2)\lambda})^{Q_I}} \cdot \prod_{s \in \{0,1\}^\lambda} \frac{1}{(2^\lambda)^{P_s}},$$

where P_s denotes the number of ideal cipher queries with key input s . Additionally, depending on oracle queries, some values may be excluded as candidates for $m_{\bar{i}}$,

$$\Pr[\gamma = T_{\text{re}}] \geq \frac{1}{(2^{(2C+2)\lambda})^{Q_I}} \cdot \prod_{s \in \{0,1\}^\lambda} \frac{1}{(2^\lambda)^{P_s}},$$

where P_s denotes the number of ideal cipher queries with key input s . Therefore, by Lemma 2, the advantage is bounded by

$$\text{Adv}_{\text{IC-VSC}}^{Q_I\text{-mih}}(\mathcal{A}) \leq \frac{(3 + 5N^2 + (2C + 8)^2 + \log N)Q_I}{2^\lambda} + \frac{(25 + 24 \log N)Q}{2^\lambda} + \frac{2}{2^\lambda}.$$

□

C Full Proof of the EUF-CMA Security

Theorem 2 (EUF-CMA Security of rBN++). *Assume that H_0 , H_1 , H_2 , ExpandH1, ExpandH2, and ExpandSalt are modeled as random oracles and that the (N, τ, λ) parameters of rBN++ are appropriately chosen where $|\mathbb{F}| = 2^\lambda$. For a PPT adversary $\mathcal{A}$ against the EUF-CMA security of rBN++ with a total of Q_{sig} signing oracle queries, Q random oracle queries, and P ideal cipher queries, there exist PPT adversaries*

- $\mathcal{B}$ against the EUF-KO security of rBN++,⁵
- $\mathcal{C}$ against the PRF security of H_3 ,⁶
- $\mathcal{D}$ against the multi-instance hiding security of IC-VSC

such that

$$\begin{aligned} \text{Adv}_{\text{rBN}++}^{\text{euf-cma}}(\mathcal{A}) &\leq \frac{(Q_{\text{sig}} + Q)^2}{2^{2\lambda}} + Q_{\text{sig}} \cdot \text{Adv}_{H_3}^{\text{prf}}(\mathcal{C}) \\ &\quad + \text{Adv}_{\text{IC-VSC}}^{\tau Q_{\text{sig}}\text{-mih}}(\mathcal{D}) + \text{Adv}_{\text{rBN}++}^{\text{euf-ko}}(\mathcal{B}), \end{aligned}$$

provided that $\tau \cdot Q_{\text{sig}} \leq \min \left\{ 2^{\frac{5(\lambda-24)}{6}}, \frac{2^{\lambda-1}}{(2C+4)N} \right\}$.

Proof. Let $\mathcal{A}$ be an EUF-CMA adversary against rBN++ for given pk . Let G_0 be the original EUF-CMA game. Let $\mathcal{O}_{\text{sig}}$ be the signing oracle, and let Q_i for $i = 0, 1, 2$ be the number of queries made to H_i by $\mathcal{A}$. Let Q_{salt} be the number of queries made to ExpandSalt by $\mathcal{A}$. We begin to prove the security of the deterministic version of rBN++ ($\rho \leftarrow 0^n$), and prove that of the probabilistic version later. Without loss of generality, we assume that all messages in signing queries are distinct.

G_1 : This game acts the same as G_0 except that it aborts if there exist two different queries on H_0 with the same outputs. This game is indistinguishable from the previous game unless the simulation is aborted. As the output length of H_0 is 2λ , we have

$$\Pr[\mathsf{G}_1 \text{ aborts}] \leq \frac{(Q_{\text{sig}} + Q_0)^2}{2^{2\lambda}}.$$

⁵ We assume $\mathcal{B}$ has the same amount of queries to random oracles.

⁶ H_3 itself is not a PRF, but it is used as a PRF with key prepending. We use this notation for convenience.

G_2 : $\mathcal{O}_{\text{sig}}$ replaces $\text{salt} \in \{0, 1\}^{2\lambda}$ with randomly sampled values, instead of computing $H_3(\text{sk}, \mu, \rho)$. As μ is always distinct for each query, the difference between this game and the previous one reduces to the PRF security of H_3 with secret key sk . Therefore, there exists a PPT adversary $\mathcal{C}$ against the PRF security of H_3 such that

$$|\Pr[\mathcal{A} \text{ wins } G_1] - \Pr[\mathcal{A} \text{ wins } G_2]| \leq Q_{\text{sig}} \cdot \mathbf{Adv}_{H_3}^{\text{prf}}(\mathcal{C}).$$

G_3 : $\mathcal{O}_{\text{sig}}$ samples $h_1 \in \{0, 1\}^{2\lambda}$ at random instead of computing

$$H_1(\mu, \text{salt}, (\text{com}_k, (\text{pk}_k^{(i)})_{i \in [N]}, \Delta c_k, (\Delta z_{k,j})_{j \in [C]})_{k \in [\tau]})$$

and programs the random oracle H_1 to output h_1 for the respective query. The first challenge $(\epsilon_{k,j})_{k \in [\tau], j \in [\ell+1]}$ is derived by expanding the randomly sampled h_1 . The simulation is aborted if the queries to H_1 have been made previously in a signing oracle query. As $\text{salt} \in \{0, 1\}^{2\lambda}$ is random, this game is indistinguishable from the previous game unless the simulation is aborted, and the probability of abort is

$$\Pr[G_3 \text{ aborts}] \leq \frac{Q_{\text{sig}}(Q_{\text{sig}} + Q_1)}{2^{2\lambda}}.$$

G_4 : $\mathcal{O}_{\text{sig}}$ now samples $h_2 \in \{0, 1\}^{2\lambda}$ at random instead of computing

$$H_2(h_1, \text{salt}, ((\alpha_k^{(i)})_{i \in [N]}, (v_k^{(i)})_{i \in [N]})_{k \in [\tau]})$$

and also program the random oracle H_2 to output h_2 for the respective query. In this game, both h_1 and h_2 are sampled in advance, and all the derived values are computed from h_1 and h_2 . After computing all such values, $\mathcal{O}_{\text{sig}}$ program the H_1 oracle and the H_2 oracle. The simulation is aborted if the queries to H_2 have been made previously in a signing oracle query. As $h_1 \in \{0, 1\}^{2\lambda}$ is random, this game is indistinguishable with the previous game unless the simulation is aborted, and the probability of abort is

$$\Pr[G_4 \text{ aborts}] \leq \frac{Q_{\text{sig}}(Q_{\text{sig}} + Q_2)}{2^{2\lambda}}.$$

G_5 : $\mathcal{O}_{\text{sig}}$ now samples $(\text{salt}_1, \dots, \text{salt}_\tau) \in \{0, 1\}^{2\tau\lambda}$ at random instead of computing $\text{ExpandSalt}(\text{salt})$, and also program the random oracle ExpandSalt to output $(\text{salt}_1, \dots, \text{salt}_\tau)$ for the respective query. The simulation is aborted if the queries to ExpandSalt have been made previously in a signing oracle query. As $\text{salt} \in \{0, 1\}^{2\lambda}$ is random, this game is indistinguishable with the previous game unless the simulation is aborted, and the probability of abort is

$$\Pr[G_5 \text{ aborts}] \leq \frac{Q_{\text{sig}}(Q_{\text{sig}} + Q_{\text{salt}})}{2^{2\lambda}}.$$

G_6 : For each $k \in [\tau]$, $\mathcal{O}_{\text{sig}}$ replaces the hidden tape $\text{tape}_k^{(\bar{i}_k)}$ by a random element in $\mathbb{F}^{2^{C+2}}$. Since $(\text{decom}_i)_{i \in [\tau]}$ is computed independently from sk , the distribution of $(\text{pdecom}_i)_{i \in [\tau]}$ is also independent to sk .

The difference between G_5 and G_6 perfectly corresponds to multi-instance hiding game of IC-VSC, so there exists a PPT adversary $\mathcal{D}$ against the multi-instance hiding game such that

$$|\Pr[\mathcal{A} \text{ wins } G_5] - \Pr[\mathcal{A} \text{ wins } G_6]| \leq \mathbf{Adv}_{\text{IC-VSC}}^{\tau Q_{\text{sig-mih}}}(\mathcal{D}).$$

We note that the multi-instance hiding game is defined within a single repetition. So the adversary $\mathcal{D}$ should have τ times more instance than the number of signature queries.

G_7 : $\mathcal{O}_{\text{sig}}$ replaces

$$(\Delta \text{sk}_k, \Delta c_k, (\Delta z_{k,j})_{j \in [C]})_{k \in [\tau]}$$

with random elements instead of computing them using sk and S-box outputs. As the hidden tapes are random, the distribution of these variables does not change.

Note that now for all $k \in [\tau]$, $(\alpha_k^{(\bar{i}_k)})_{k \in [\tau]}$ is random and independent of sk . If the multiplication triple is wrong, then $v_k^{(\bar{i}_k)} \leftarrow -\sum_{i \neq \bar{i}_k} v_k^{(i)}$ is different from an honest value derived from legitimate calculation. However, $(\bar{i}_k)$ is unopened and the multiplication check is still passed. Since the signature oracle in G_7 does not depend on the secret key sk , it implies that G_7 can be reduced to the EUF-KO security. Therefore, there exists a PPT adversary $\mathcal{B}$ on EUF-KO security against rBN++ such that

$$\Pr[\mathcal{A} \text{ wins } G_7] \leq \mathbf{Adv}_{\text{rBN++}}^{\text{euf-ko}}(\mathcal{B}).$$

All in all, we have

$$\begin{aligned} \mathbf{Adv}_{\text{rBN++}}^{\text{euf-cma}}(\mathcal{A}) &\leq \frac{(Q_{\text{sig}} + Q_0)^2}{2^{2\lambda}} + Q_{\text{sig}} \cdot \mathbf{Adv}_{H_3}^{\text{prf}}(\mathcal{C}) \\ &\quad + \frac{Q_{\text{sig}}(Q_{\text{sig}} + Q_1)}{2^{2\lambda}} + \frac{Q_{\text{sig}}(Q_{\text{sig}} + Q_2)}{2^{2\lambda}} + \frac{Q_{\text{sig}}(Q_{\text{sig}} + Q_{\text{salt}})}{2^{2\lambda}} \\ &\quad + \mathbf{Adv}_{\text{IC-VSC}}^{\tau Q_{\text{sig-mih}}}(\mathcal{D}) + \mathbf{Adv}_{\text{rBN++}}^{\text{euf-ko}}(\mathcal{B}) \\ &\leq \frac{(Q_{\text{sig}} + Q)^2}{2^{2\lambda}} + Q_{\text{sig}} \cdot \mathbf{Adv}_{H_3}^{\text{prf}}(\mathcal{C}) \\ &\quad + \mathbf{Adv}_{\text{IC-VSC}}^{\tau Q_{\text{sig-mih}}}(\mathcal{D}) + \mathbf{Adv}_{\text{rBN++}}^{\text{euf-ko}}(\mathcal{B}) \end{aligned}$$

provided that $Q_0 + Q_1 + Q_2 + Q_{\text{salt}} \leq Q$.

For the non-deterministic version of $\mathcal{A}$, all games are defined in a manner almost identical to the deterministic version, with the exception of handling two queries to $\mathcal{O}_{\text{sig}}$ that involve the same messages and ρ values. If (m, ρ) are identical in two queries, the outputs must also be identical; thus, we avoid random sampling and use already programmed outputs for the random oracles in such cases. Consequently, the differences between the adjacent games remain unchanged from the deterministic version, leading to the same bounds on the advantage of $\mathcal{A}$. $\square$

D Application of One-tree Technique.

Recently, Baum et al. introduced *batched all-but-one vector commitment (BAVC)* [2] which consists of

1. a single large GGM tree containing all the τN seeds,
2. a proof-of-work mechanism.

For the first one, the single large GGM tree generates all the τN seeds and reveals all-but- τ seeds. We leave the application of the first technique as a future work.

For the second one, the last challenge hash (H_2 in the rBN++ scheme) checks whether the last w bits are all zero. If the bits are not all zero, the prover calls the hash once more with an increased counter, which is called *proof-of-work*. The prover includes the counter in the signature for the verifier to verify without the proof-of-work. Fortunately, this technique can be directly applied to the rBN++ scheme. In Table 3, we summarize the increased number of random oracle calls and reduced signature size for a reasonable amount of proof-of-work. Unlike FAEST, since the rBN++ scheme calls almost no random oracle per signature, there may be some computation overhead.

N	τ	w	RO call	IC call	Sig. size
16	33	0	5	4191	5376
16	32	4	21	4064	5218
16	31	8	271	3937	5058
256	17	0	5	34799	3904
256	16	8	271	32752	3682

Table 3: The number of calls to the random oracles and the ideal cipher for rAIMer when the proof-of-work is applied. w is the number of bits for the proof-of-work. We assume that the counter is 2-byte long.